



Introduction to Machine Learning Methods

Learning in a dynamic environment

Haris Gačanin
RWTH Aachen University

Outline

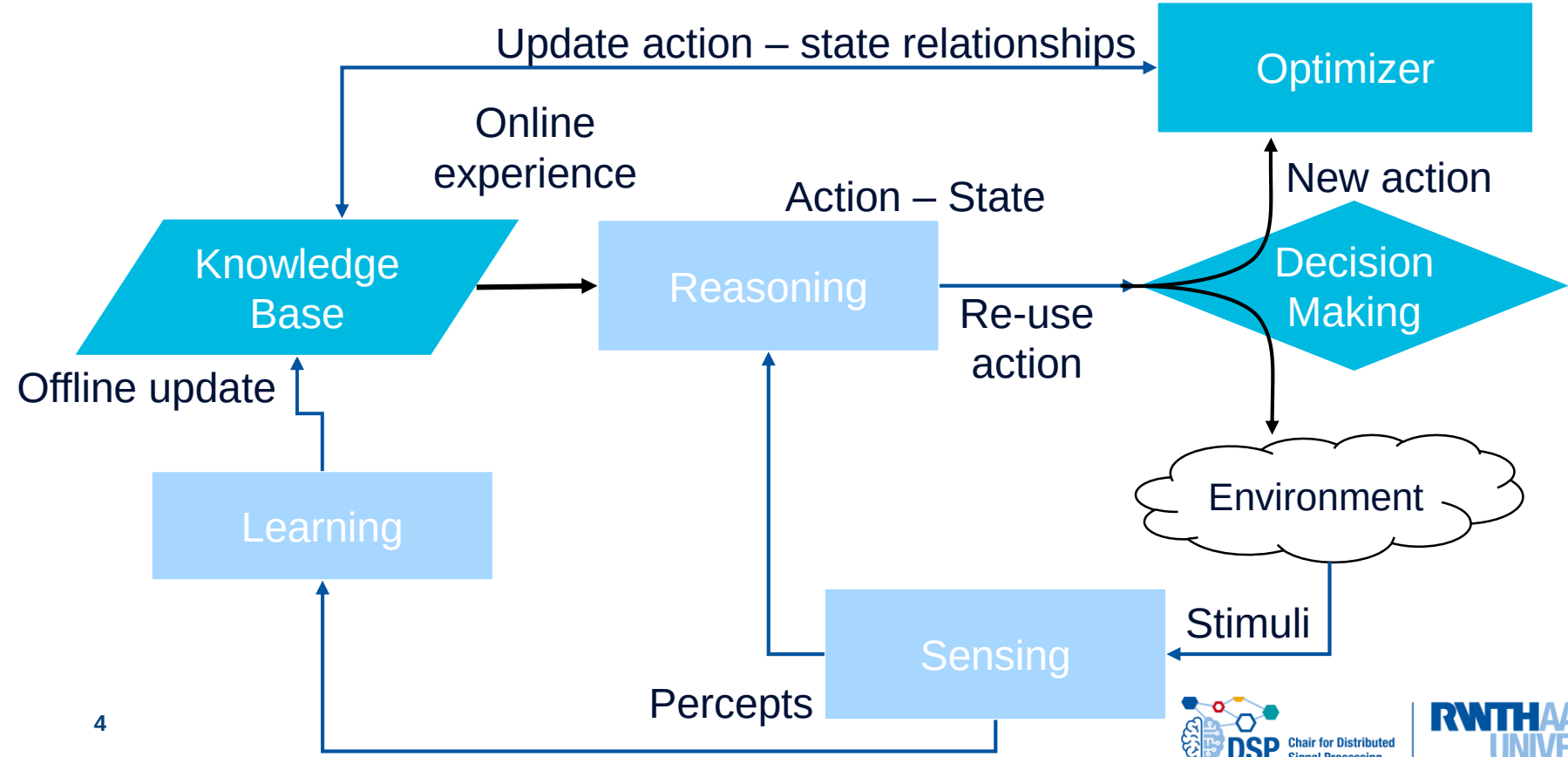
- RL agent example
- Learning (optimizer design)
- Deep RL



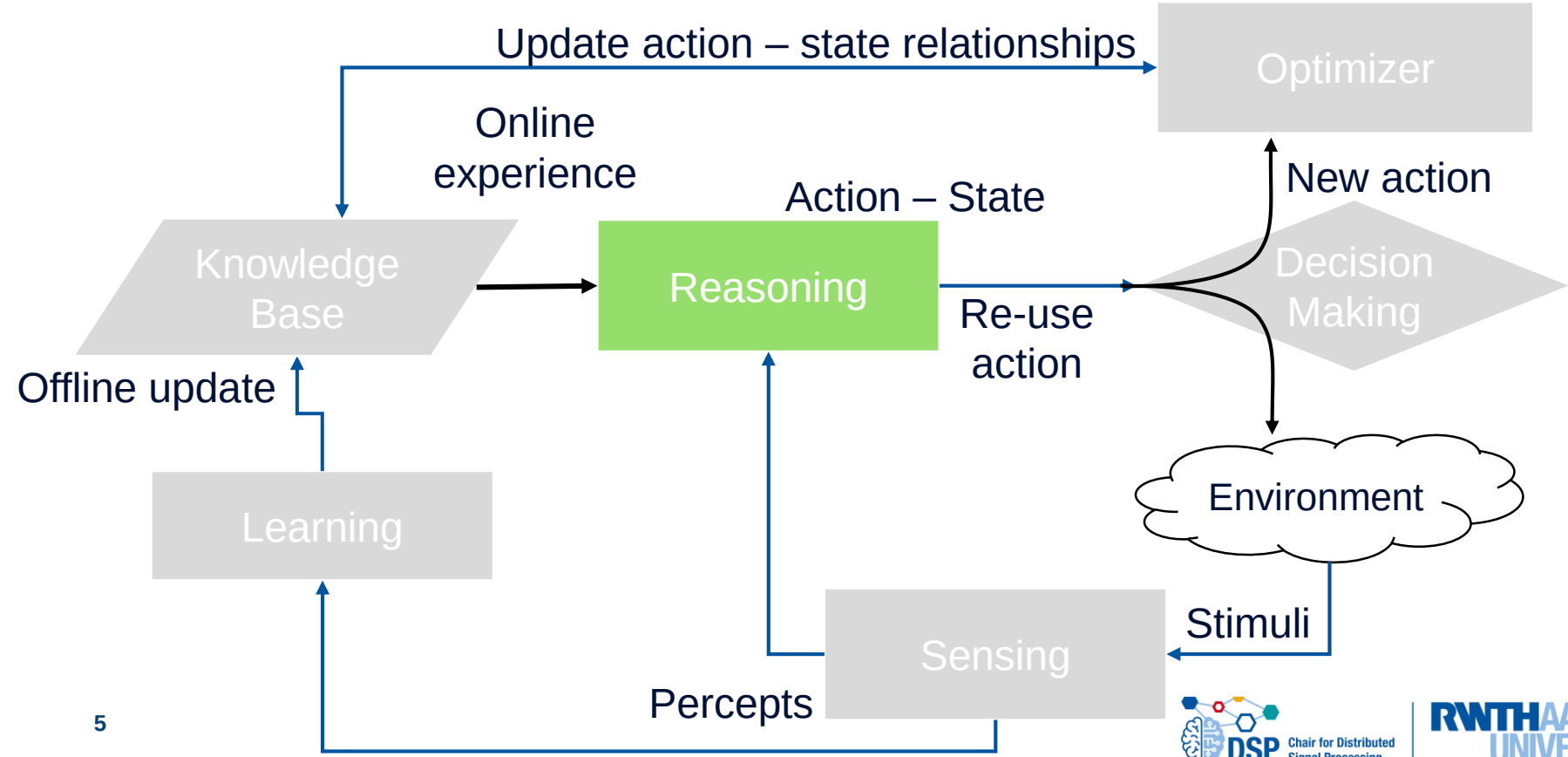
Learning agent functions

Functions that we need to implement

Computational agent



Computational agent - Reasoning

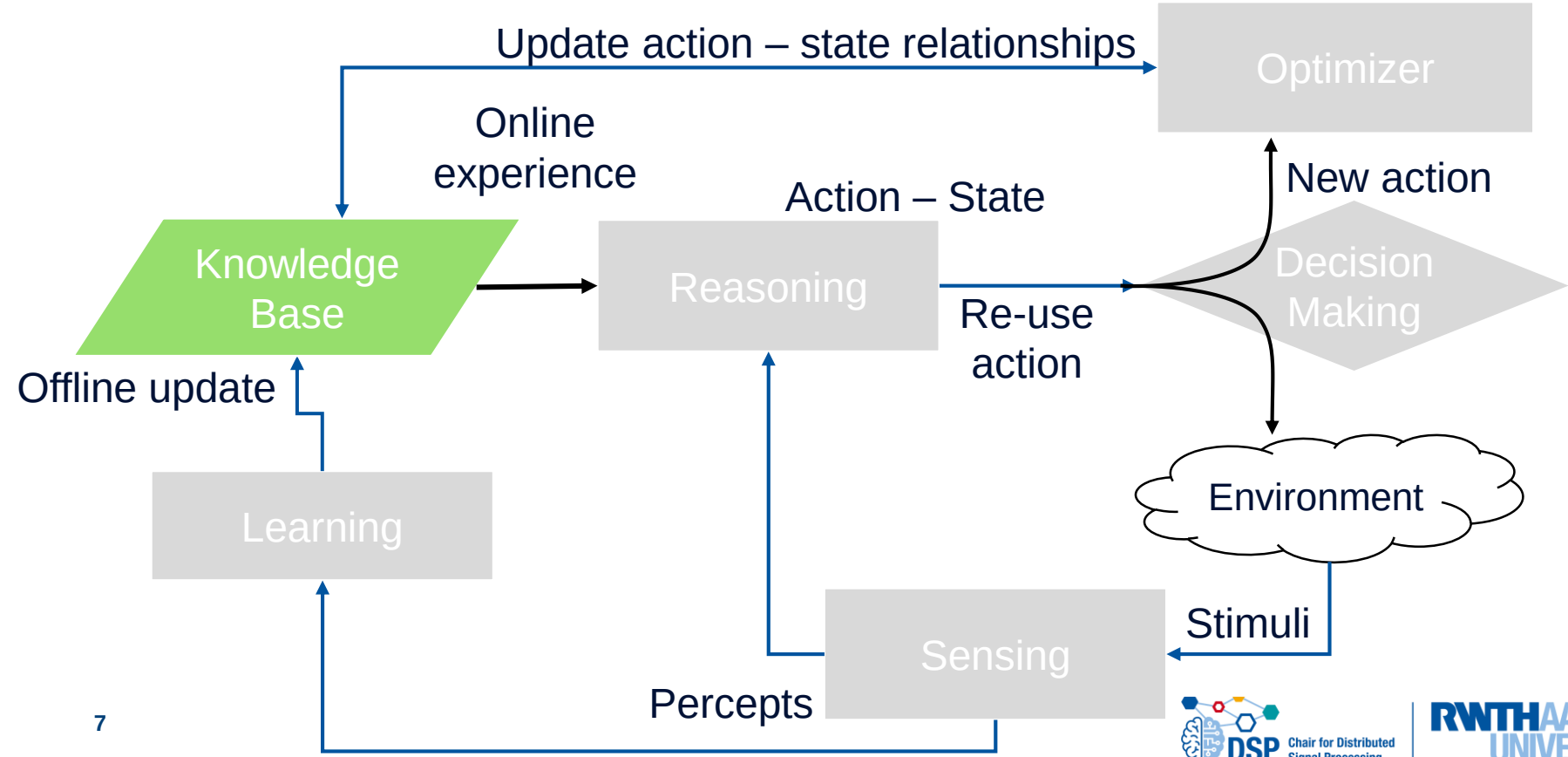


Reasoning

- Syntax
- Semantics

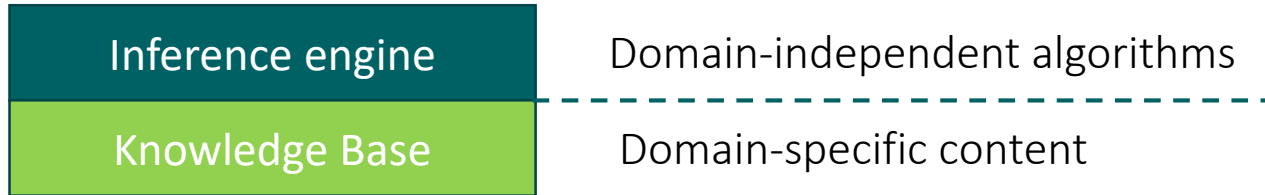


Computational agent – Knowledge base



Knowledge base

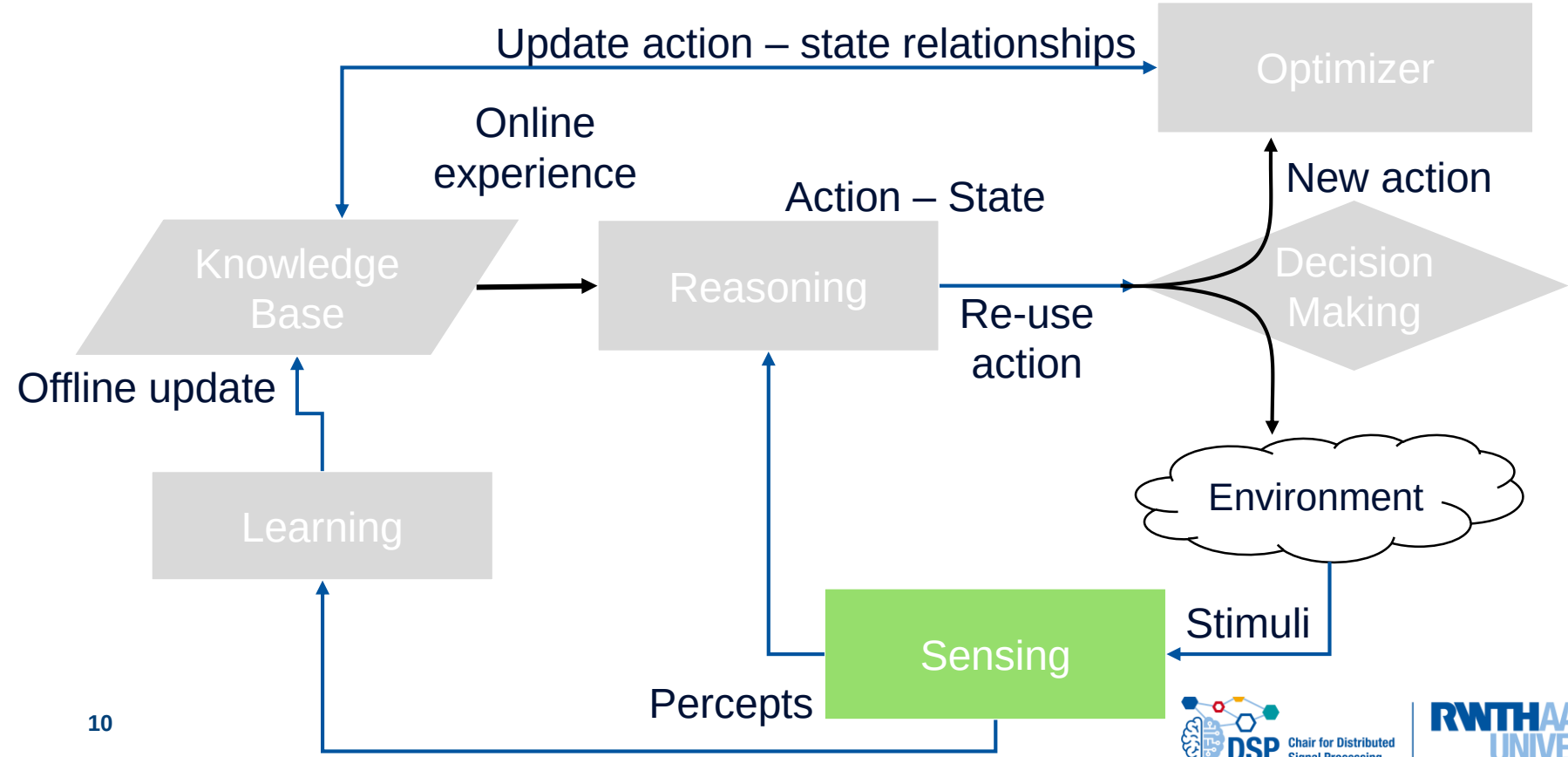
- Declarative approach to building an agent
 - Tell it what it needs to know
 - Agent asks itself what to do - answers should follow from the KB



A simple knowledge-based agent

- The agent must be able to:
 - Represent states, actions, etc.
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions
- A knowledge-based system is a system that uses knowledge about a domain to act or to solve problems.

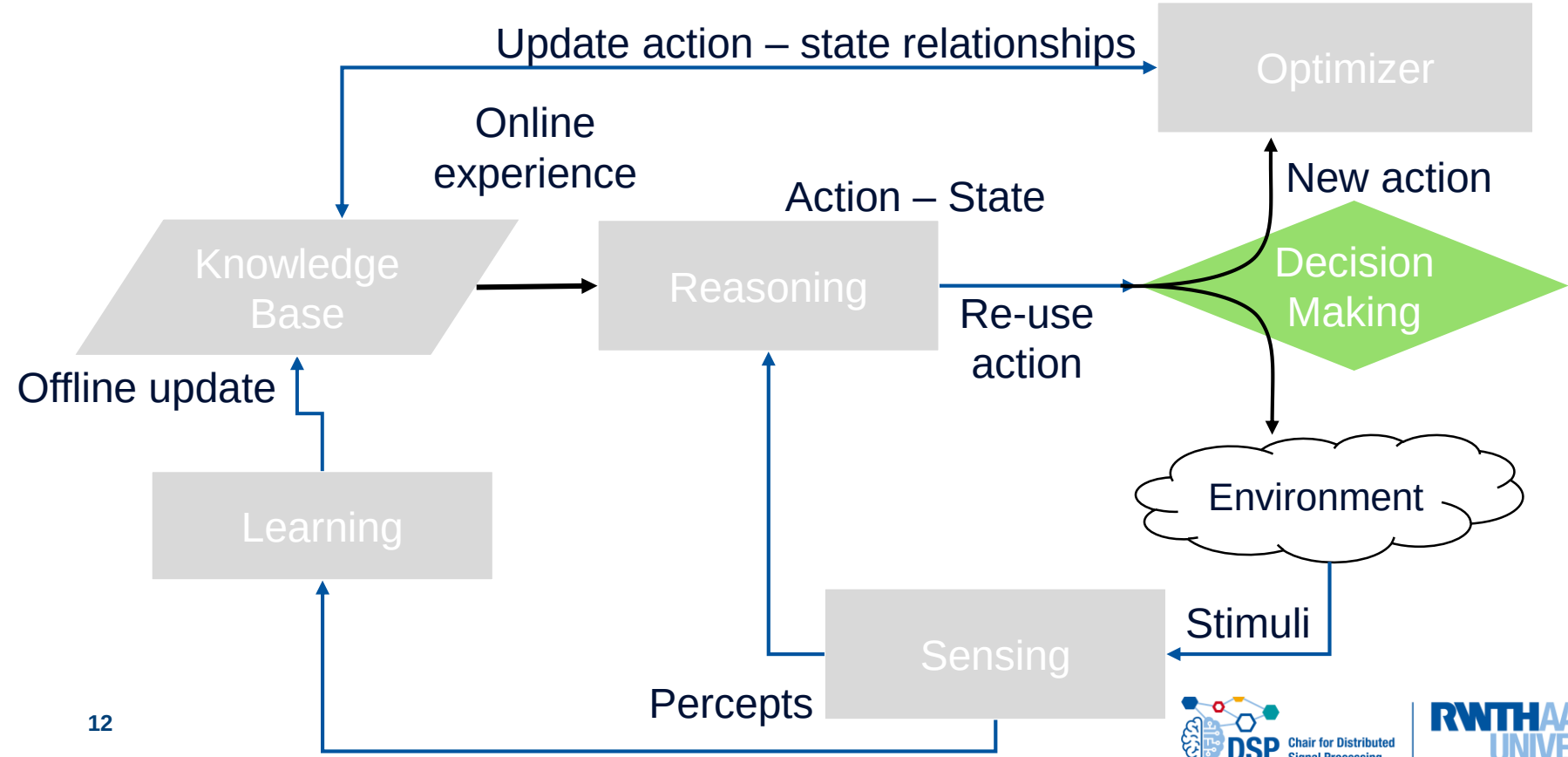
Computational agent - Reasoning



Sensing of environment

- Embedded sensors
 - Lidars
 - Cameras
 - Temperature sensors
- Logical sensors
 - Measurements (i.e., measuring A and B)
 - Combining two measurements, like A/B

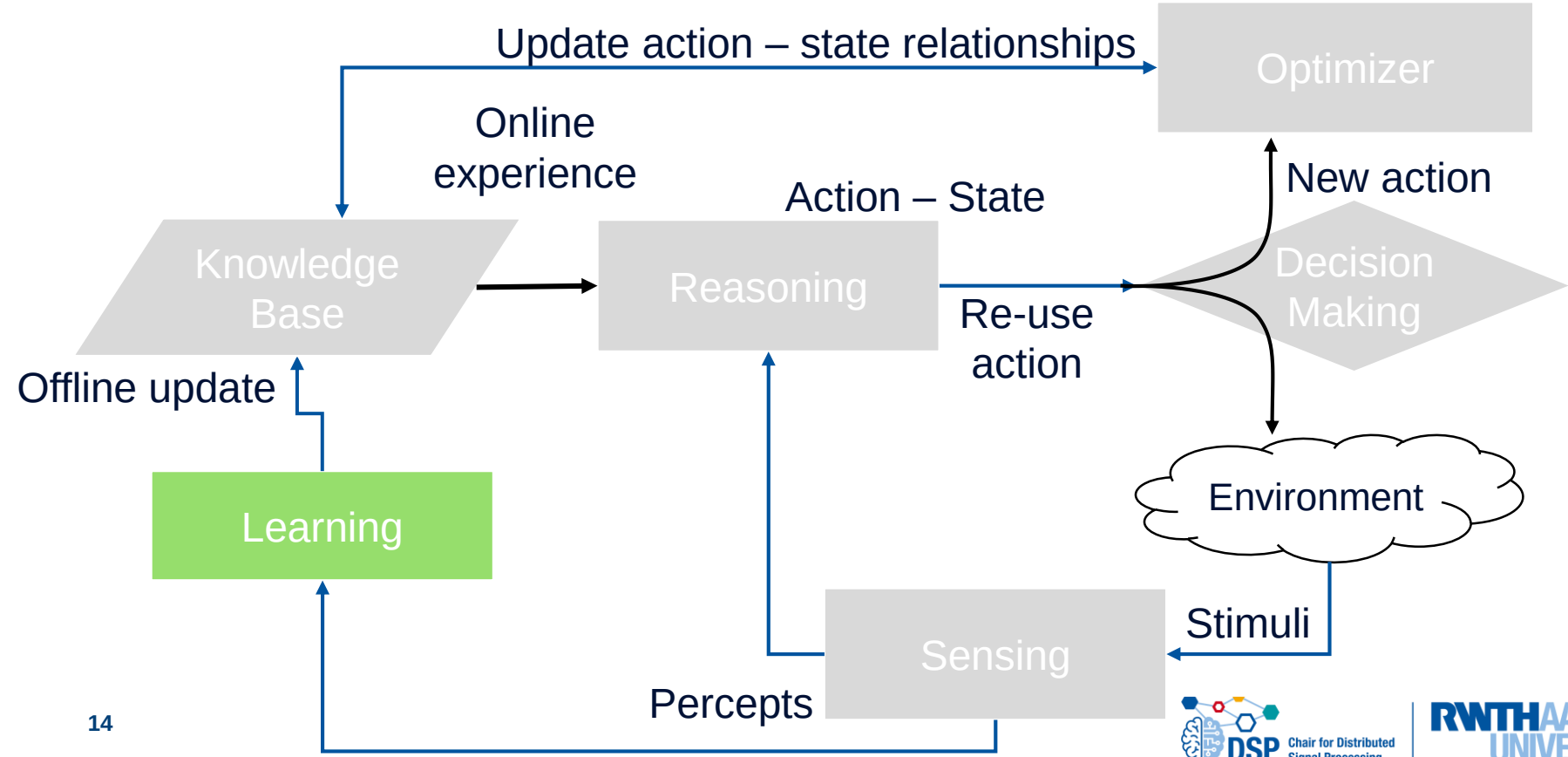
Computational agent – Decision making



Decision making

- Deterministic
- Sequential
- Probabilistic

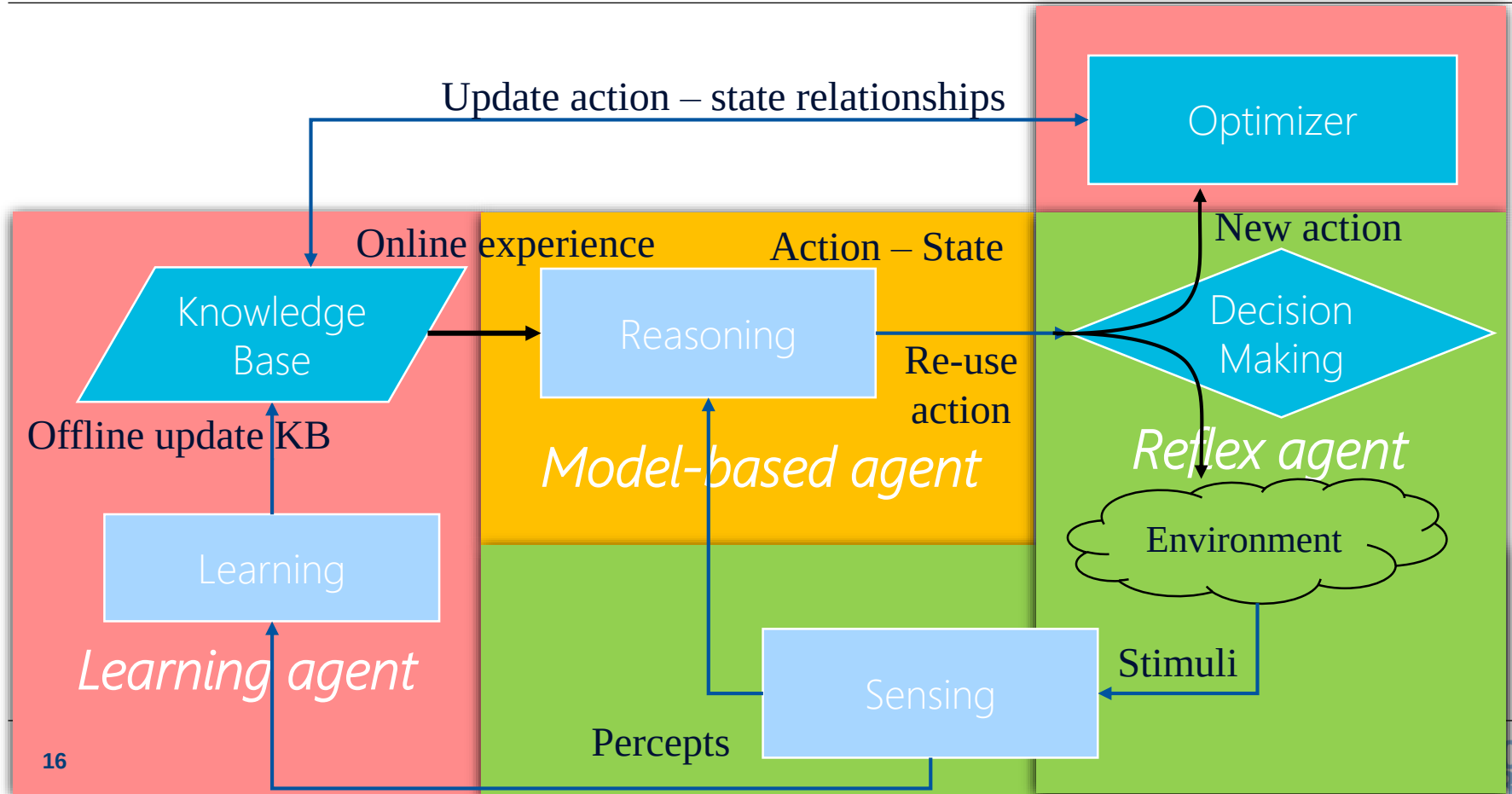
Computational agent – Learning



Learning agents

- Learning from examples (supervised or inductive learning)
- Unsupervised learning
- Reinforcement learning
- Probabilistic

Summary

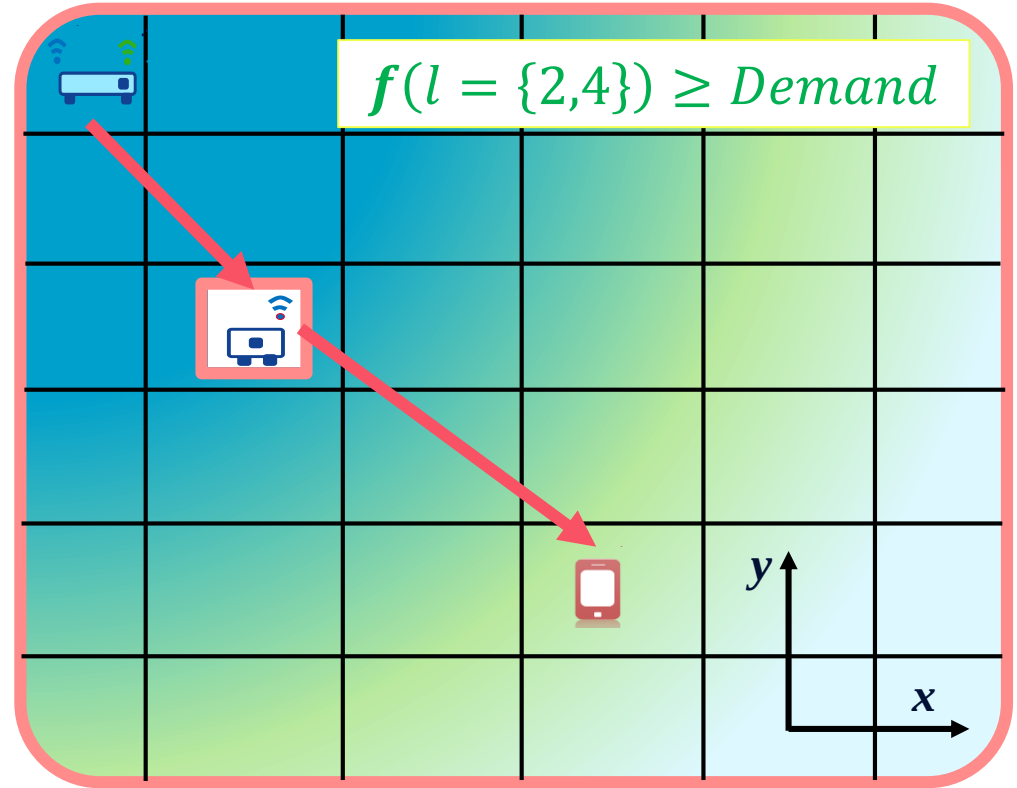




An example

Location search problem

$t=0$

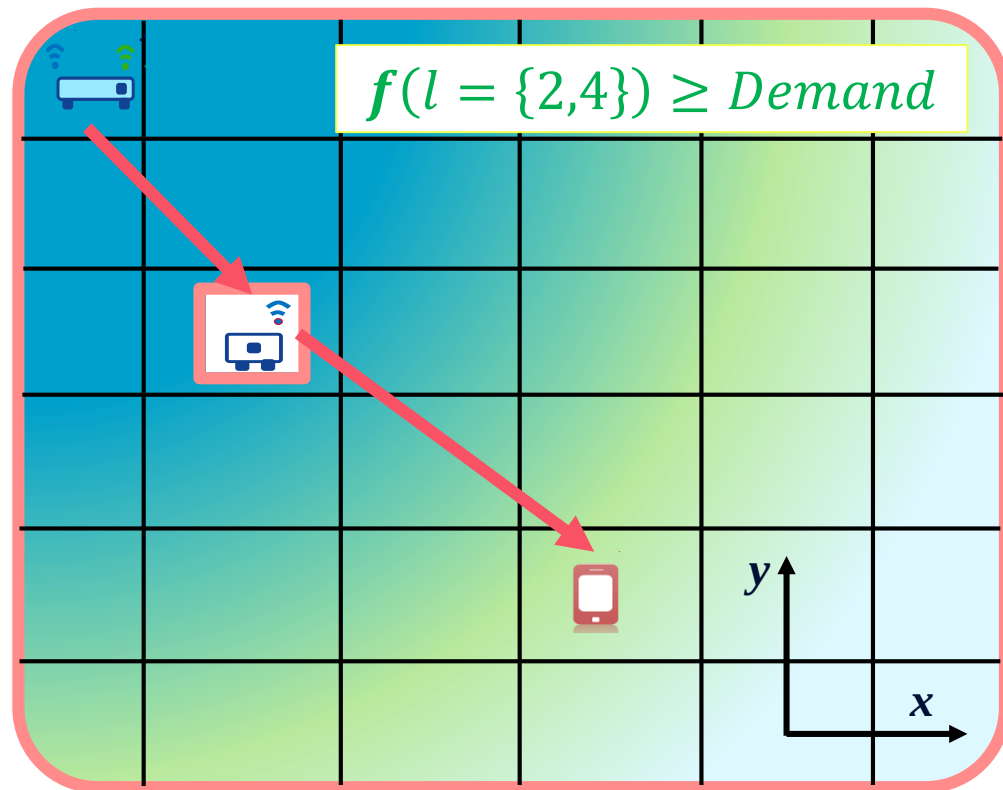


Location search problem

Objective: Max.
throughput (f)

$$\arg \max_{l \in L} f(l)$$

$t=0$

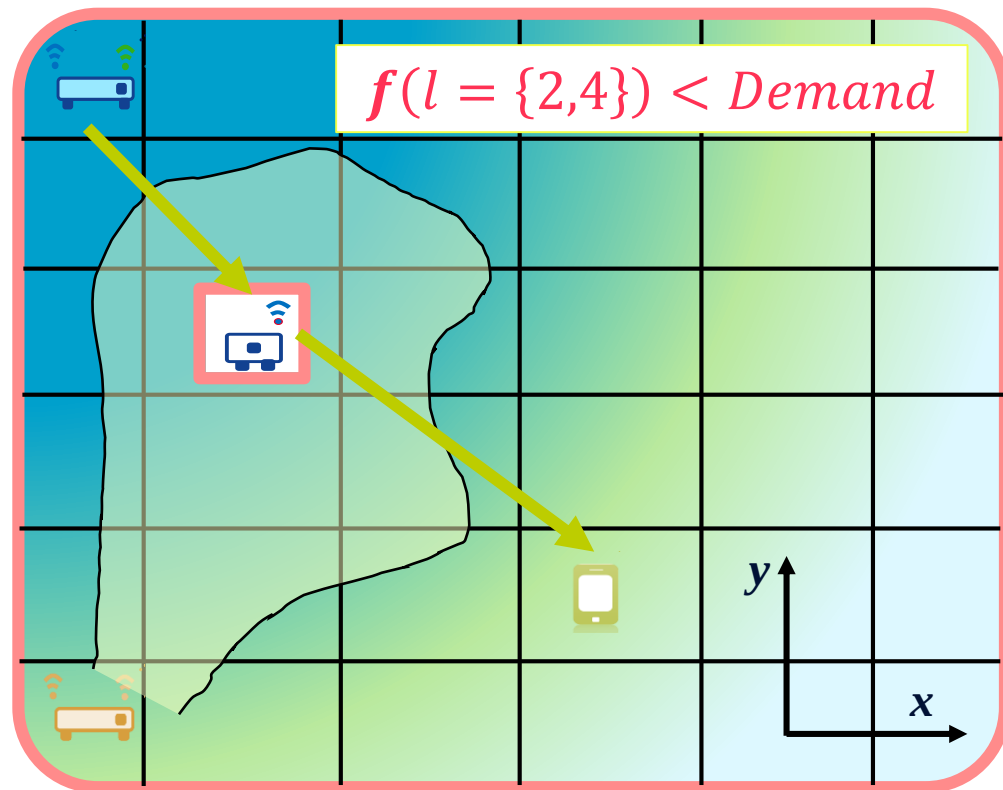


Location search problem

Objective: Max.
throughput (f)

$$\arg \max_{l \in L} f(l)$$

$t=1$

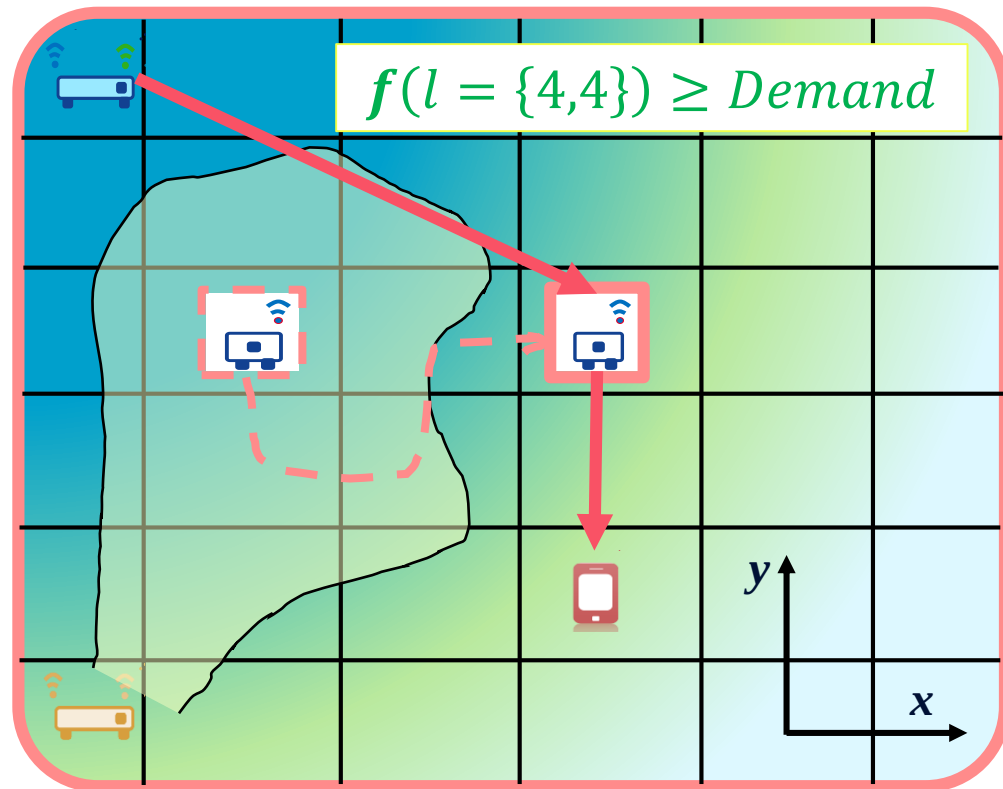


Location search problem

Objective: Max.
throughput (f)

$$\arg \max_{l \in L} f(l)$$

$t=2$

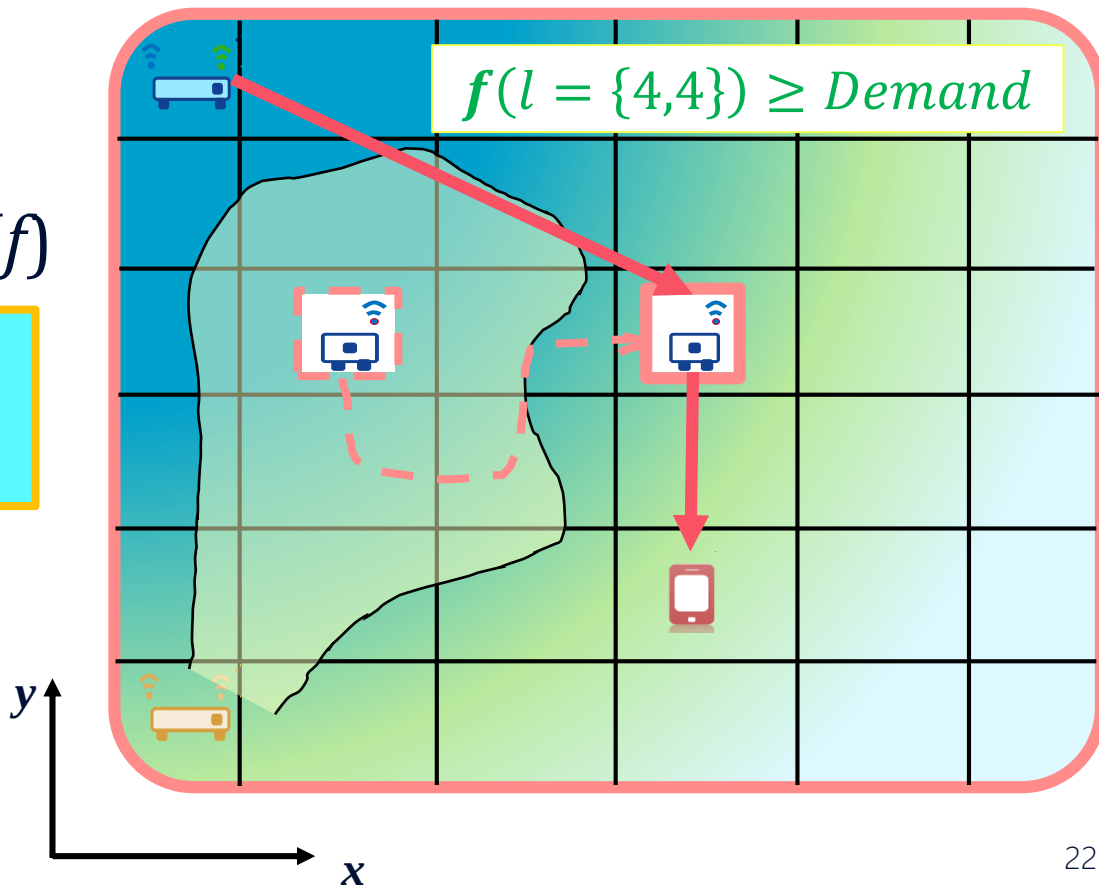


Location search problem

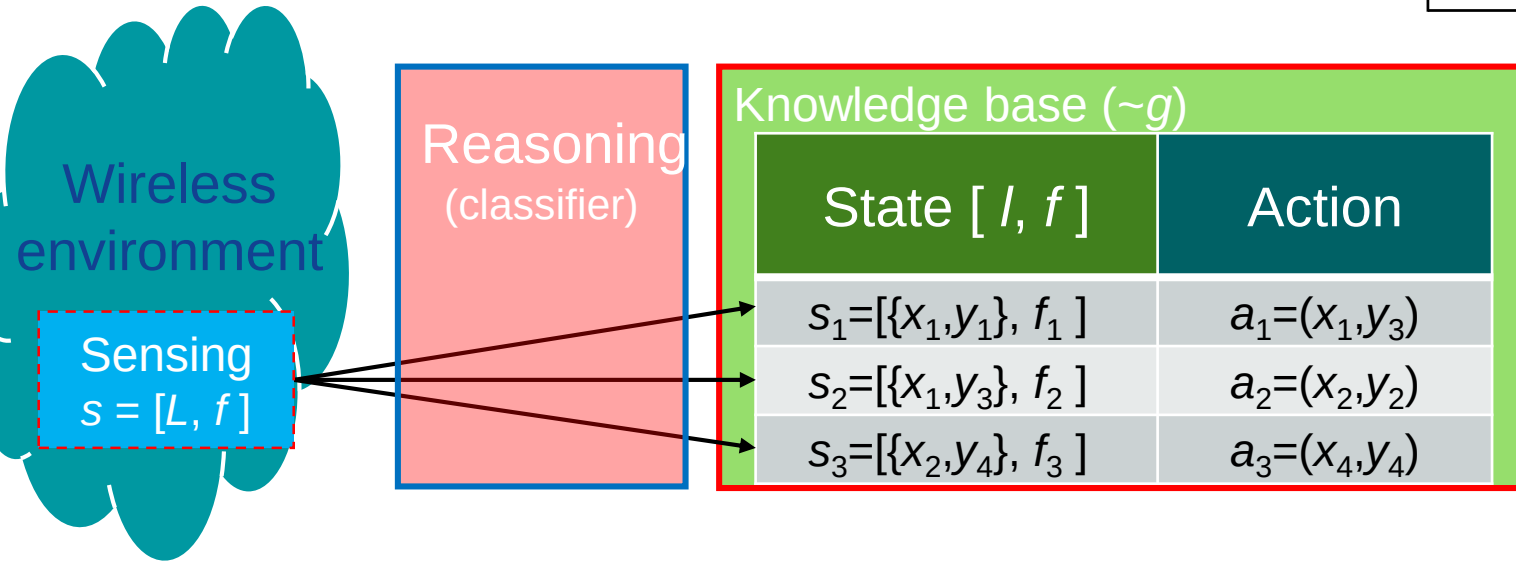
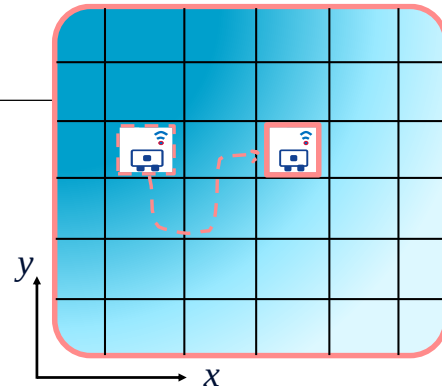
Objective: Max.
throughput (f)

$$\arg \max_{l \in L} f(l)$$

$t=2$

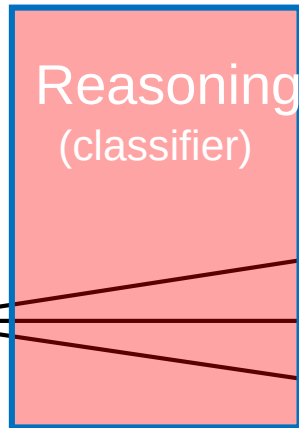
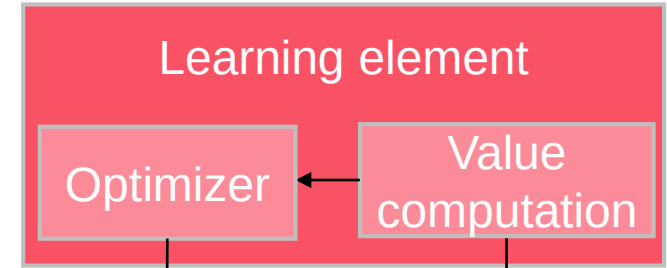


Agent components



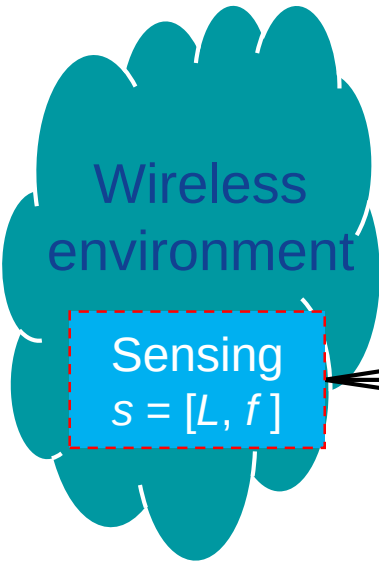
Learning agent components

$$Q(a, s) = \frac{f(l = \{x, y\})}{Demand}$$



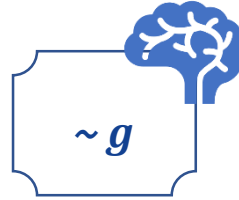
Knowledge base ($\sim g$)

State [l, f]	Action	Quality
$s_1 = [\{x_1, y_1\}, f_1]$	$a_1 = (x_1, y_3)$	$Q(a_1, s_1) (=0.7)$
$s_2 = [\{x_1, y_3\}, f_2]$	$a_2 = (x_2, y_2)$	$Q(a_2, s_2) (=0.9)$
$s_3 = [\{x_2, y_4\}, f_3]$	$a_3 = (x_4, y_4)$	$Q(a_3, s_3) (=0.1)$



Knowledge base

Agent **asks** itself what to
do with answers
following from the KB

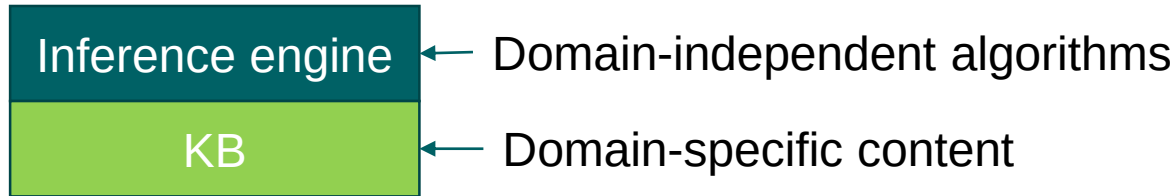


Tells the agent
what it needs
to know about
an action

Knowledge base		
State	Action	Quality
s_1 =Experience 1	a_1	$Q(a_1, s_1)$
s_2 =Experience 2	a_2	$Q(a_2, s_2)$
s_3 =Experience 3	a_3	$Q(a_3, s_3)$

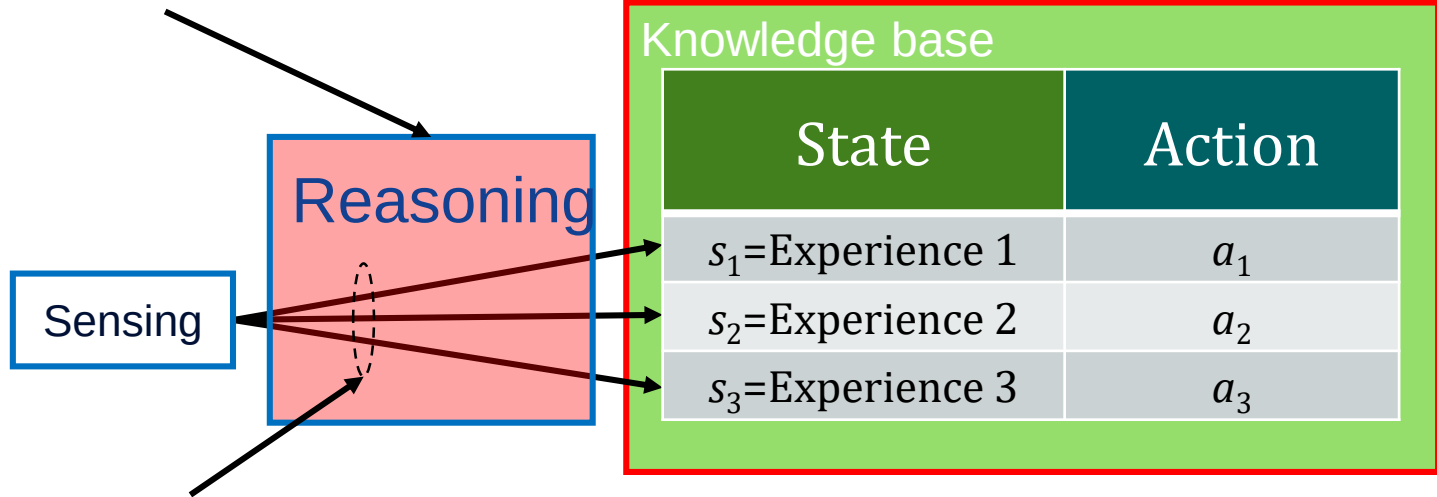
Knowledge base

- **Knowledge base** = set of **sentences** in **a formal** language
- **Declarative** approach to building an agent (or other system):
 - **Tell** it what it needs to know
- Then it can **Ask** itself what to do - answers should follow from the KB
- Agents can be viewed at the **knowledge level**
i.e., **what they know**, regardless of how implemented
- Or at the **implementation level**
 - i.e., data structures in KB and algorithms that manipulate them



Reasoning (1/2)

Manipulates with entries
of the knowledge base



Searches for the best action

Reasoning (2/2)

- Specifies *what* needs to be done, but *not how*.
- **Manipulates with entries** of the KB
- **Searches** for the best action with respect to the current state

Reasoning = Inference (information) on the top of knowledge base (data)

Learning

Exposes the agent
to environment

Essential for dynamic
environments

Learning

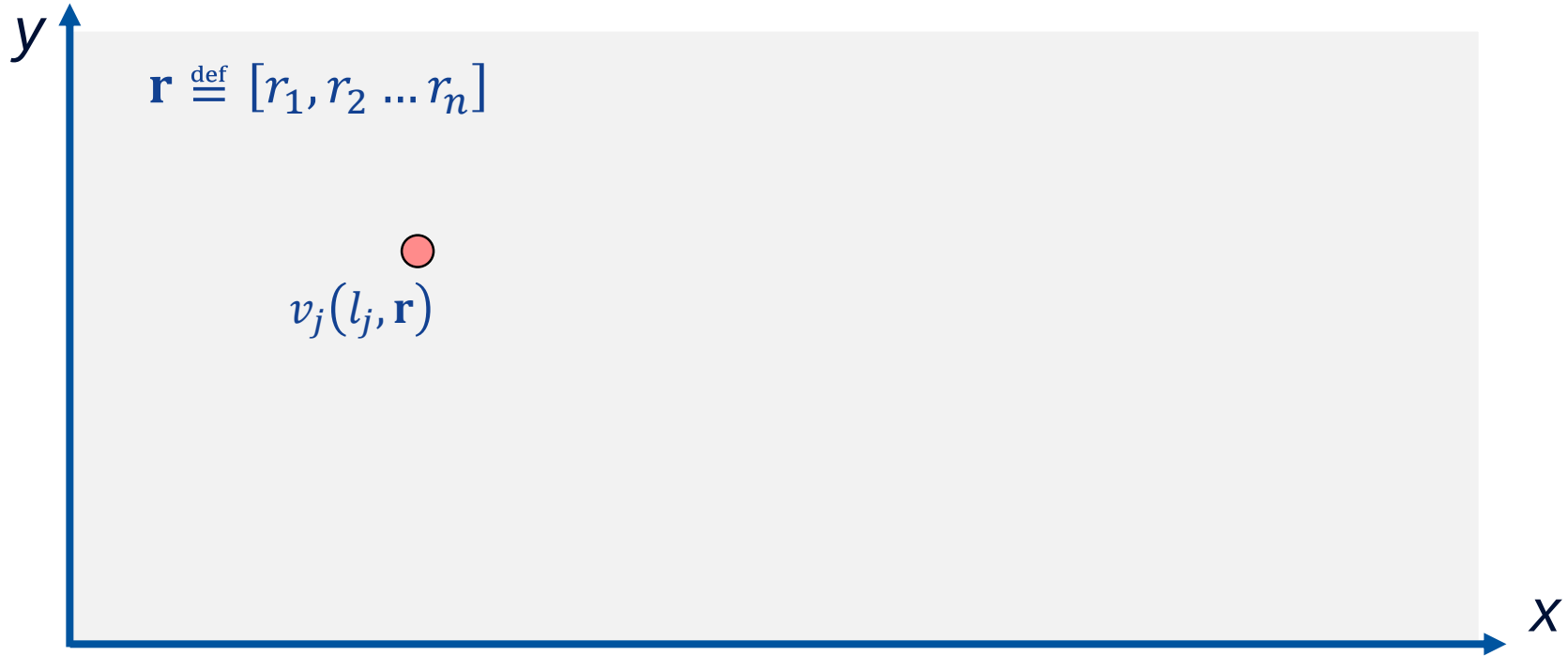
Knowledge base

State	Action	Quality
s_1 =Experience 1	a_1	$Q(a_1, s_1)$
s_2 =Experience 2	a_2	$Q(a_2, s_2)$
s_3 =Experience 3	a_3	$Q(a_3, s_3)$

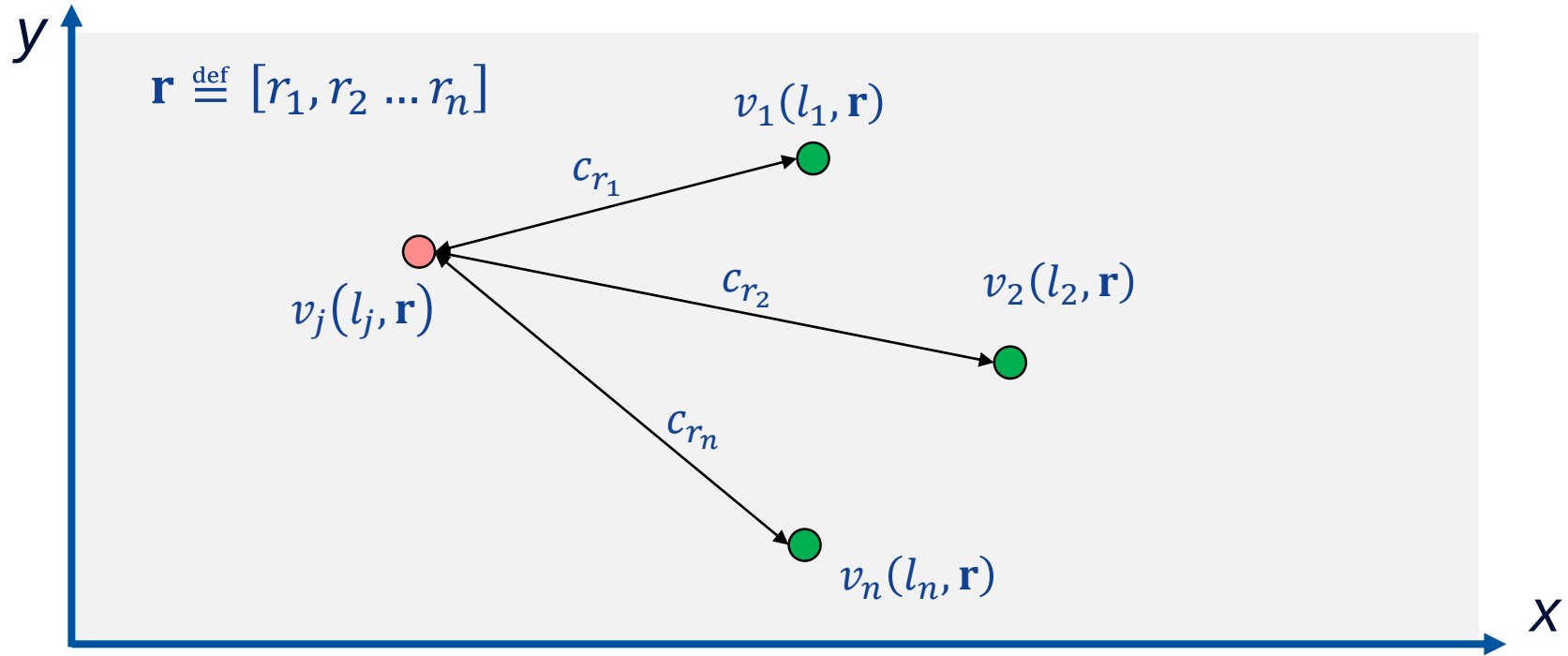


Optimizer: Exploration vs. exploitation

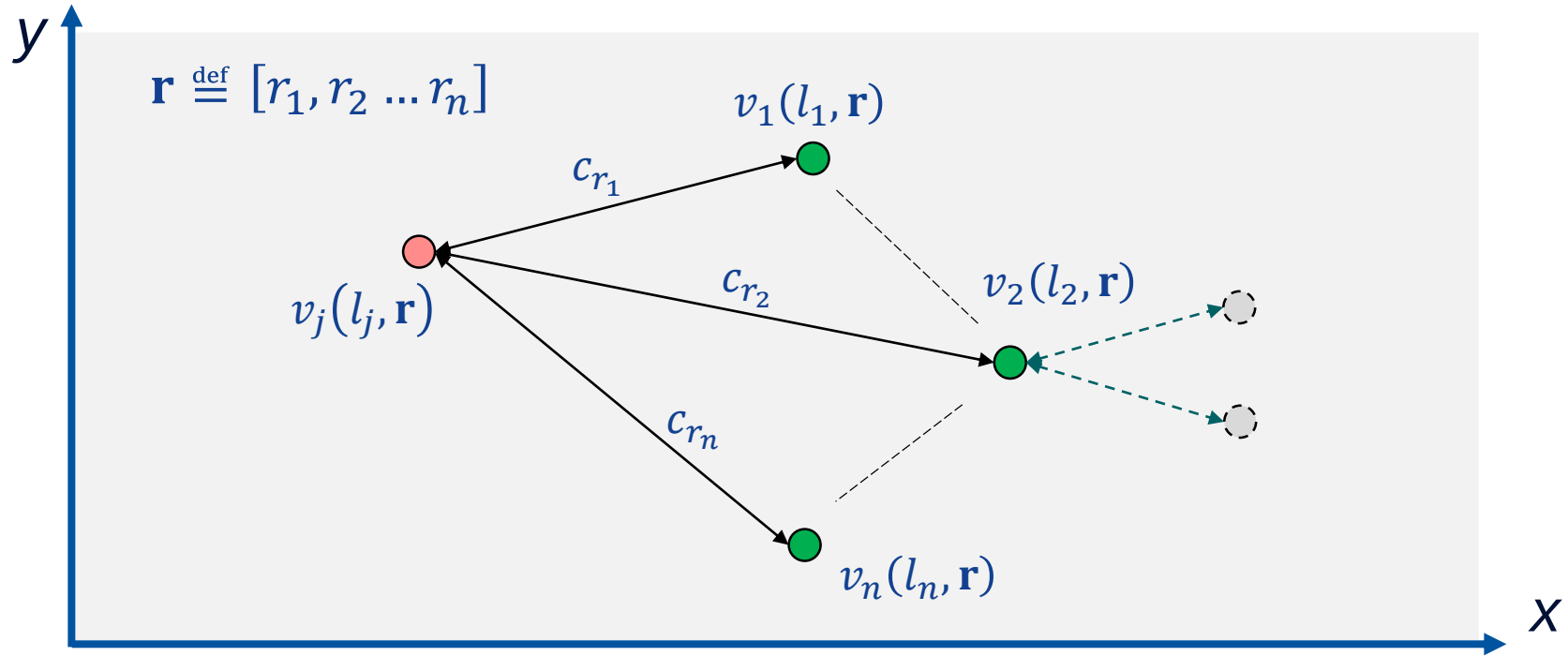
Relationship between action and environment



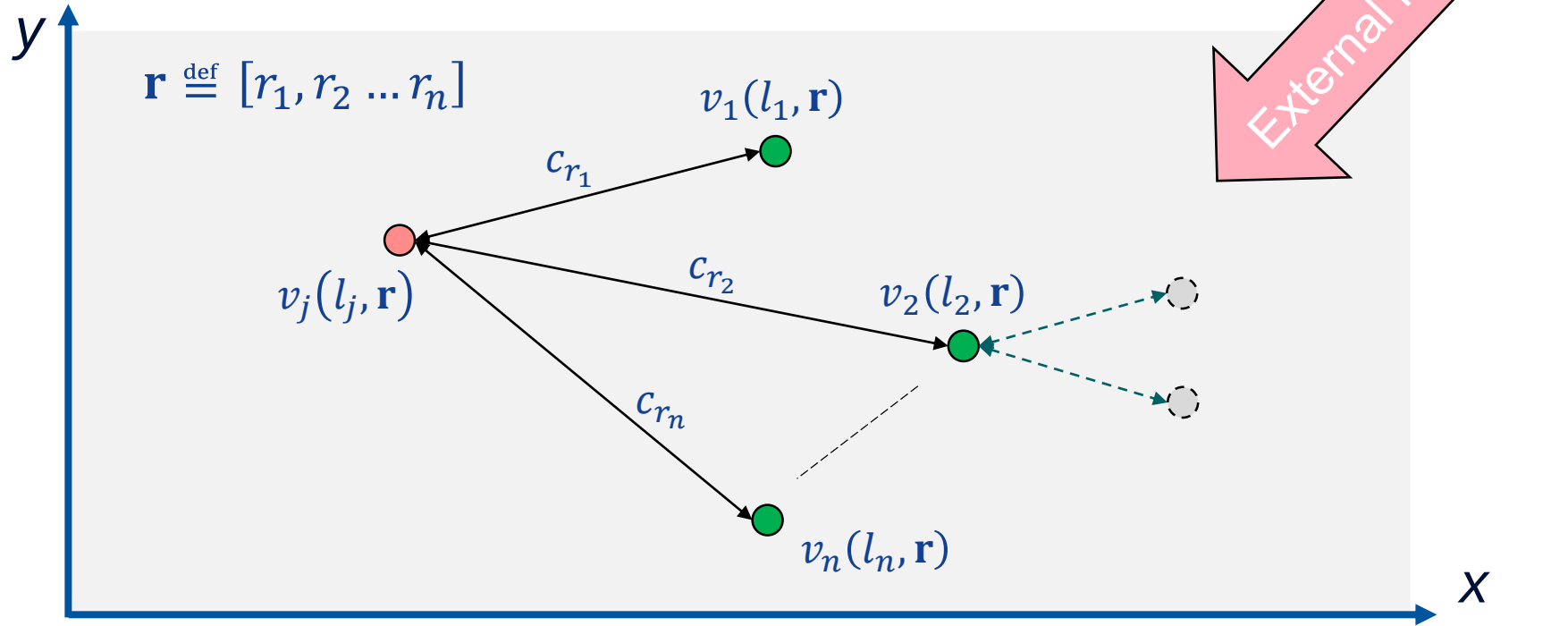
Relationship between action and environment



Relationship between action and environment



Relationship between action and environment



Greedy optimizer

$$Q(a, s) = \frac{f(l = \{x, y\})}{Demand}$$

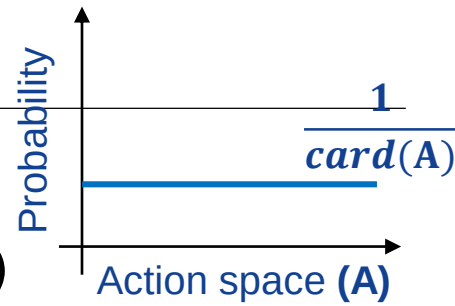
- Exploiting, No memory
- Search through quality values, Pre-defined experiences

$$a^* \leftarrow \text{Greedy}(\mathbf{A}, s) = \underset{a \in \mathbf{A}}{\operatorname{argmax}} Q(a, s)$$

$$\mathbf{A} \stackrel{\text{def}}{=} [a_1, a_2 \dots a_{n=\text{card}(\mathbf{A})}]$$

Random optimizer

- Undirected exploration, No experiences, No memory
- Agent takes “random” actions without utility (optimality)



$$a^* \leftarrow \text{Uniform}(\mathbf{A})$$

$$\mathbf{A} \stackrel{\text{def}}{=} [a_1, a_2 \dots a_{n=\text{card}(\mathbf{A})}]$$

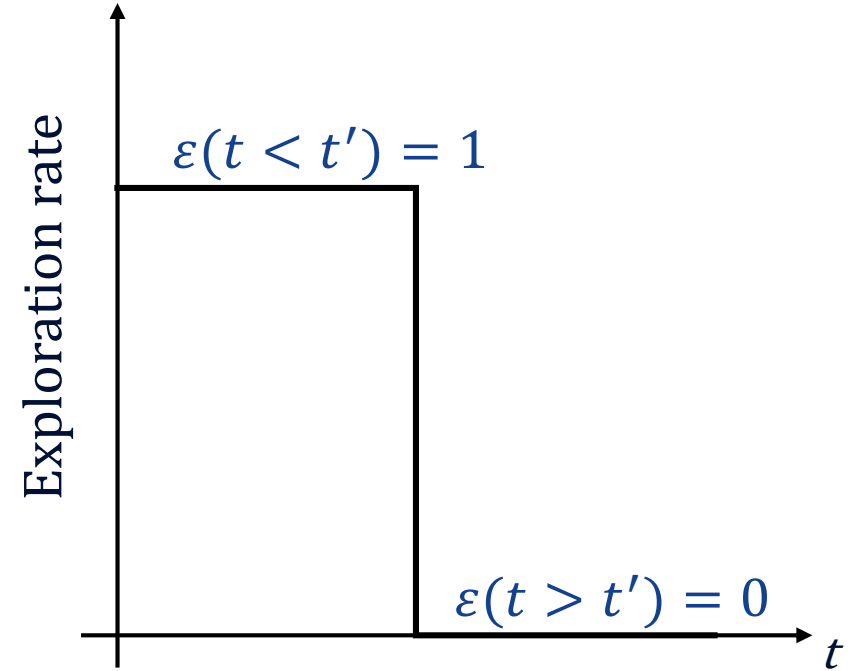
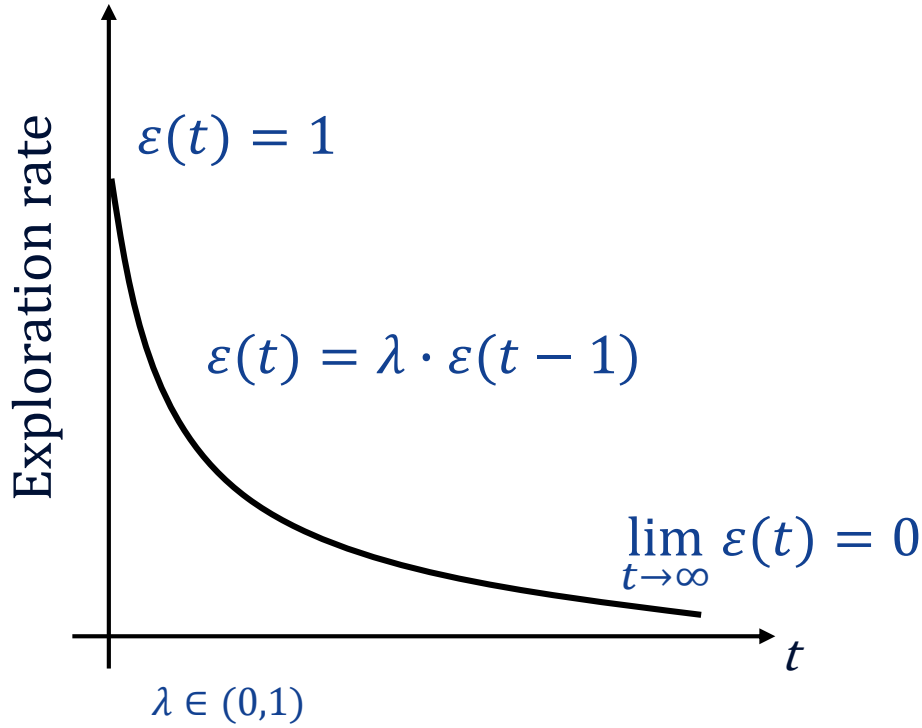
ϵ -greedy optimizer

- New parameter ϵ

$$\mathbf{A} \stackrel{\text{def}}{=} [a_1, a_2 \dots a_{n=\text{card}(\mathbf{A})}]$$

$$a^* \leftarrow \begin{cases} \text{Uniform}(\mathbf{A}); & \text{explore with prob. } \epsilon(t) \\ \text{Greedy}(\mathbf{A}, s); & \text{exploit otherwise} \end{cases}$$

Decreasing- ε and ε -first



Softmax (softargmax) random optimizer

- For $\tau \rightarrow \infty$, all actions have nearly the same probability (explore)
- Lower τ , actions with higher Q-values return higher probabilities
- For $\tau \rightarrow 0^+$, action with the highest $Q(a, s)$ is selected

$$a^* \leftarrow \text{Softmax}(\mathbf{A}, s) = \operatorname{argmax}_{a \in \mathbf{A}} \frac{e^{\frac{Q(a, s)}{\tau}}}{\sum_b e^{\frac{Q(b, s)}{\tau}}}$$

$$\mathbf{A} \stackrel{\text{def}}{=} [a_1, a_2 \dots a_{n=\text{card}(\mathbf{A})}]$$

Max-Boltzmann random optimizer

$$\mathbf{A} \stackrel{\text{def}}{=} [a_1, a_2 \dots a_{n=\text{card}(\mathbf{A})}]$$

$$a^* \leftarrow \begin{cases} \text{Softmax}(\mathbf{A}, s); & \text{explore with prob. } \varepsilon(t) \\ \text{Greedy}(\mathbf{A}, s); & \text{exploit otherwise} \end{cases}$$

Two parameters: τ, ε

Learning cost example of wireless random optimizer

Re-association delay ~s up to ~min

```
Trial>> set_channels('192.168.6.1','root','','radio1',1)
ssh root@192.168.6.1 "change-channel-all radio1 1 HT20 1 HT20"
Total delay (channel switching delay + EXT reassociation delay) is 35.64 seconds
Trial>> set_channels('192.168.6.1','root','','radio1',4)
ssh root@192.168.6.1 "change-channel-all radio1 4 HT20 4 HT20"
Total delay (channel switching delay + EXT reassociation delay) is 29.62 seconds
```

Channel switching delay 1-3 sec

```
root@LEDE_ExtenderTpLink_ap: /usr/bin
root@LEDE_ExtenderTpLink_ap:/usr/bin# ch-switchin-delay
Channel switching delay
/usr/bin/change_channel: line 12: 'radio0': not found
Total time in milliseconds: 2717
```

A problem-specific knowledge base

	v_0	v_1	...	v_M
Q table	a_1	a_2	...	a_M
Perception table	b_1	b_2	...	b_M
Environment table	D			

A problem-specific knowledge base

	v_0	v_1	...	v_M
Q table	a_1	a_2	...	a_M
Perception table	b_1	b_2	...	b_M
Environment table	D			

Percepts	Radio 1	Radio 2	Radio 3
Next hop	v_0	v_3	Infrastructure
Channel	1	5	10
$\Delta_u(\%)$	+40	-10	0
$\Delta_\rho(\%)$	+10	+20	-10
$\Delta_{retr.}(\%)$	-	-	+2
$\Delta_{err.}(\%)$	-	-	0

A problem-specific knowledge base

	v_0	v_1	...	v_M
Q table	a_1	a_2	...	a_M
Perception table	b_1	b_2	...	b_M
Environment table	D			

Percepts	Radio 1	Radio 2	Radio 3
Next hop	v_0	v_3	Infrastructure
Channel	1	5	10
$\Delta_u(\%)$	+40	-10	0
$\Delta_\rho(\%)$	+10	+20	-10
$\Delta_{retr.}(\%)$	-	-	+2
Utilization [%]	Loc 1	...	Loc L
Ch. 1	20		40
Ch. 11	-		30

A problem-specific knowledge base

Q-values (Mbps)		Channel Configuration Actions					Repositioning Actions		
		(1,1,1)	...	(4,1,2)	...	(11,11,11)	Loc 1	...	Loc L
States	Loc 1	23		50		2	-		15
	...						50		15
	Loc L	0		10		15	50		-

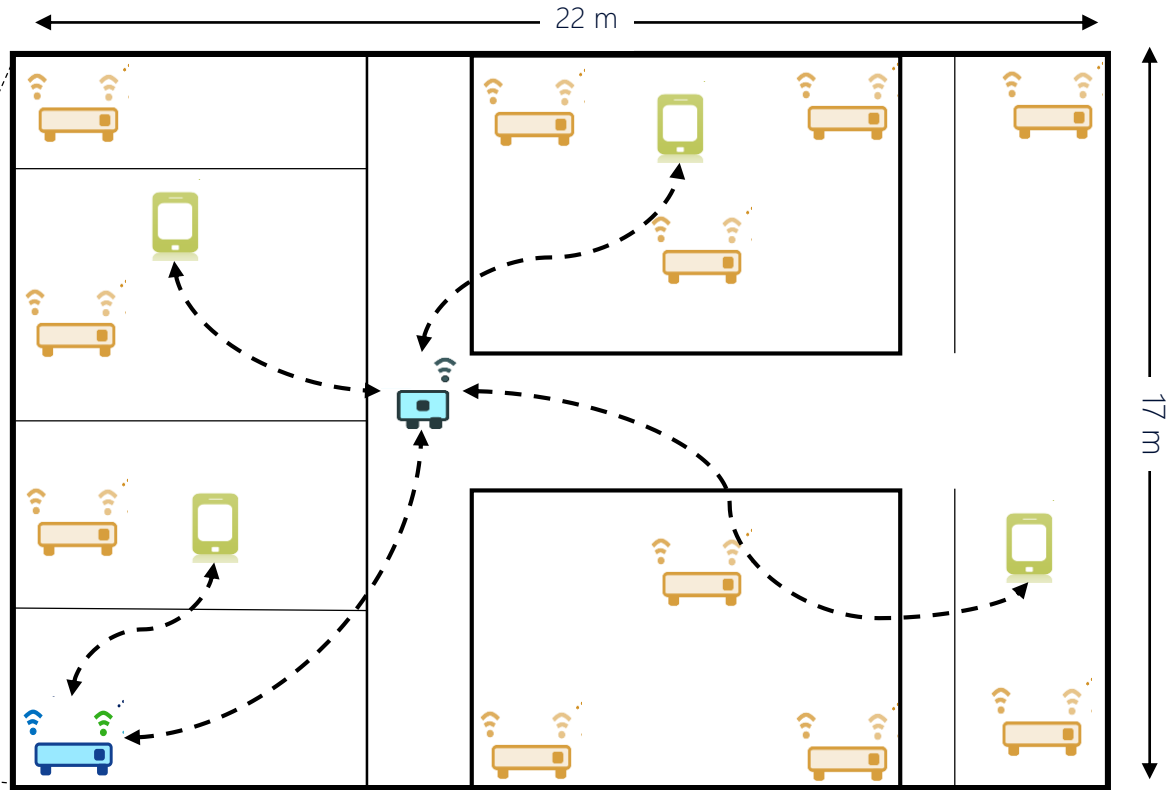
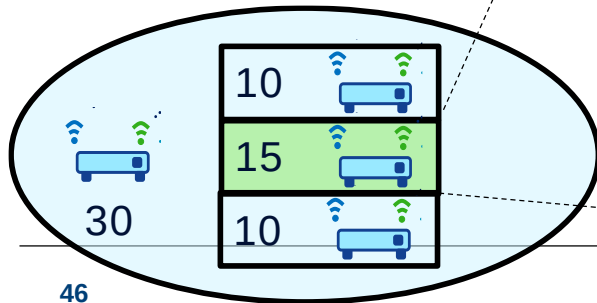
	v_0	v_1	...	v_M
Q table	a_1	a_2	...	a_M
Perception table	b_1	b_2	...	b_M
Environment table	D			

Percepts	Radio 1	Radio 2	Radio 3
Next hop	v_0	v_3	Infrastructure
Channel	1	5	10
$\Delta_v(\%)$	+40	-10	0
$\Delta_\rho(\%)$	+10	+20	-10
$\Delta_{retr.}(\%)$	-	-	+2
$\Delta_{\text{other}}(\%)$	-	-	-

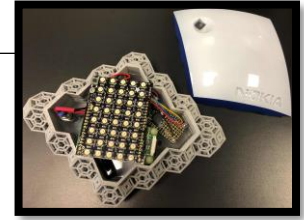
Utilization [%]	Loc 1	...	Loc L
Ch. 1	20		40
Ch . 11	-		30

Optimizer objective and prototype setup

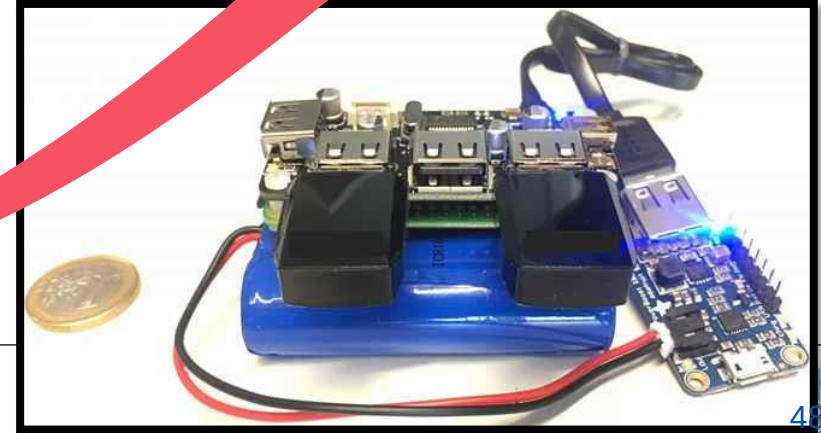
$$\arg \max_{l, c_r} \sum_u f_u(l, c_r)$$



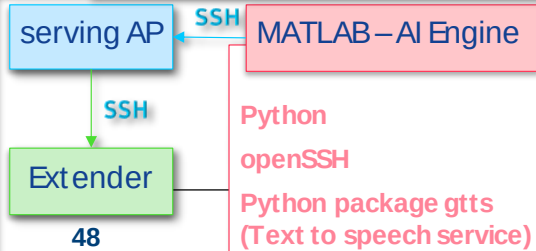
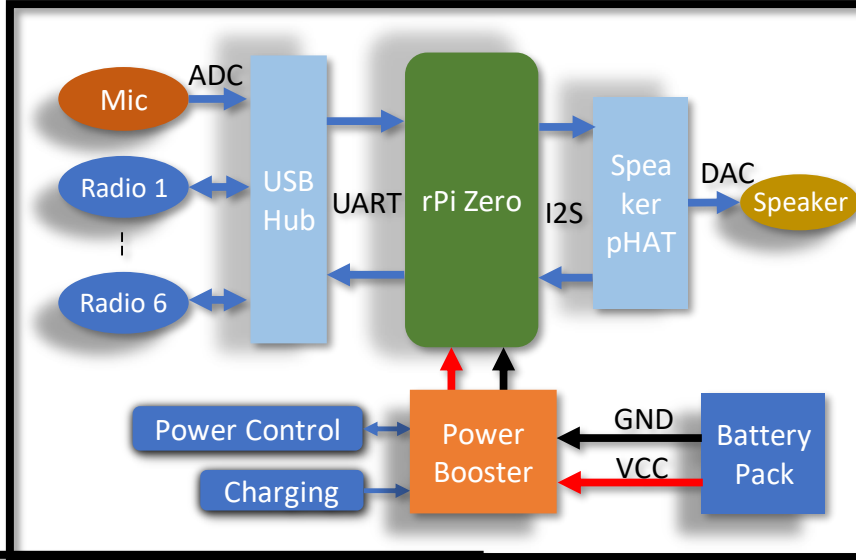
Development of radio node



2017



2015



Python
openSSH
Python package gtts
(Text to speech service)

Remarks

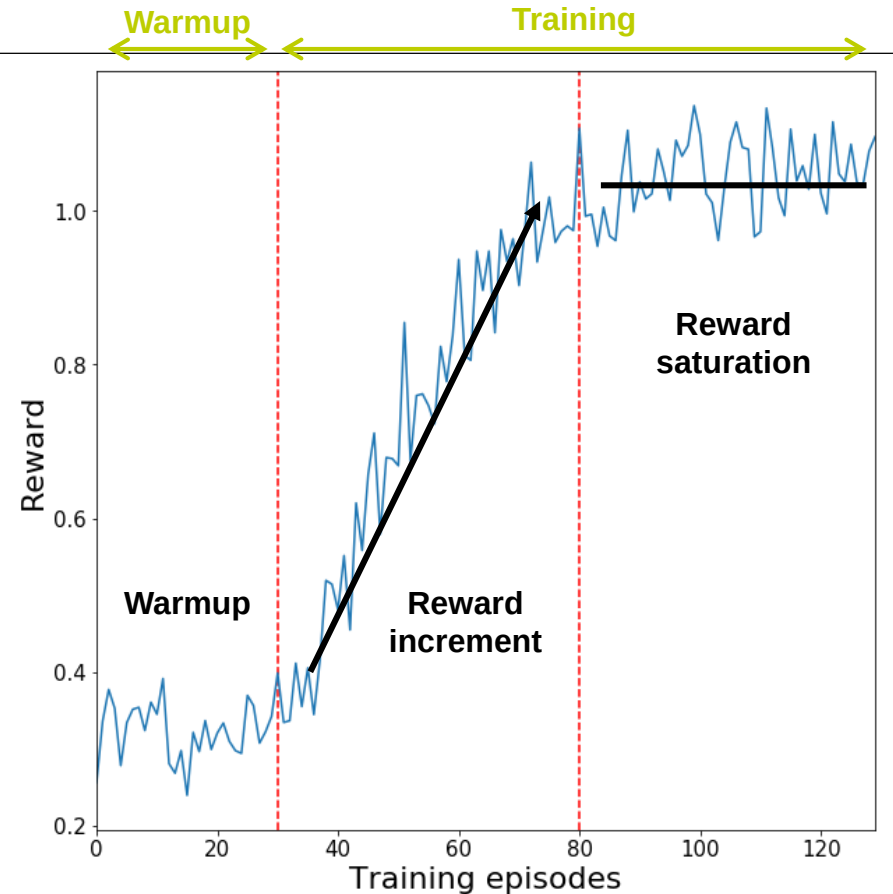
- Goal is to learn utility values of states and an optimal mapping from states to actions.
- Direct Utility Estimation ignores dependencies among states:
→ use Bellman equation.
- Temporal difference updates values to match those of successor states.
- Active reinforcement learning learns the optimal mapping from states to actions



Deep reinforcement learning

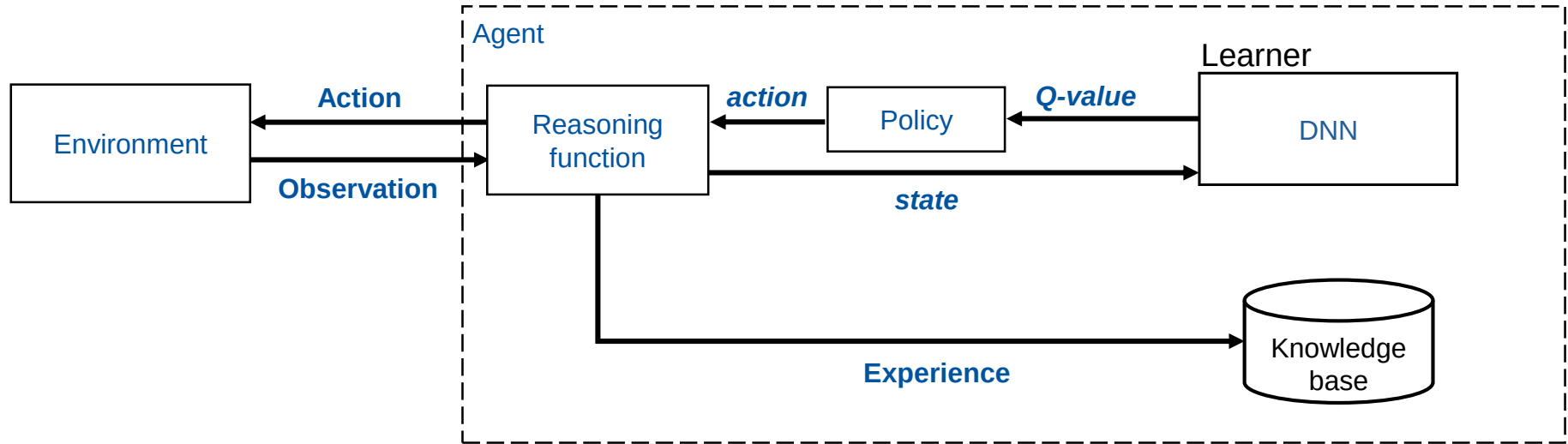
DQN - Phases

- DQN has **online training**
 - Training is done together with the prediction cycle **intermittently**.
- Warmup phase:
 - Only Exploration
- Training phase
 - Both Exploration and Exploitation
 - Increment in reward through experience replay
 - Has two parts
 - Reward Increment
 - Reward Saturation
- Prediction phase
 - Only exploitation



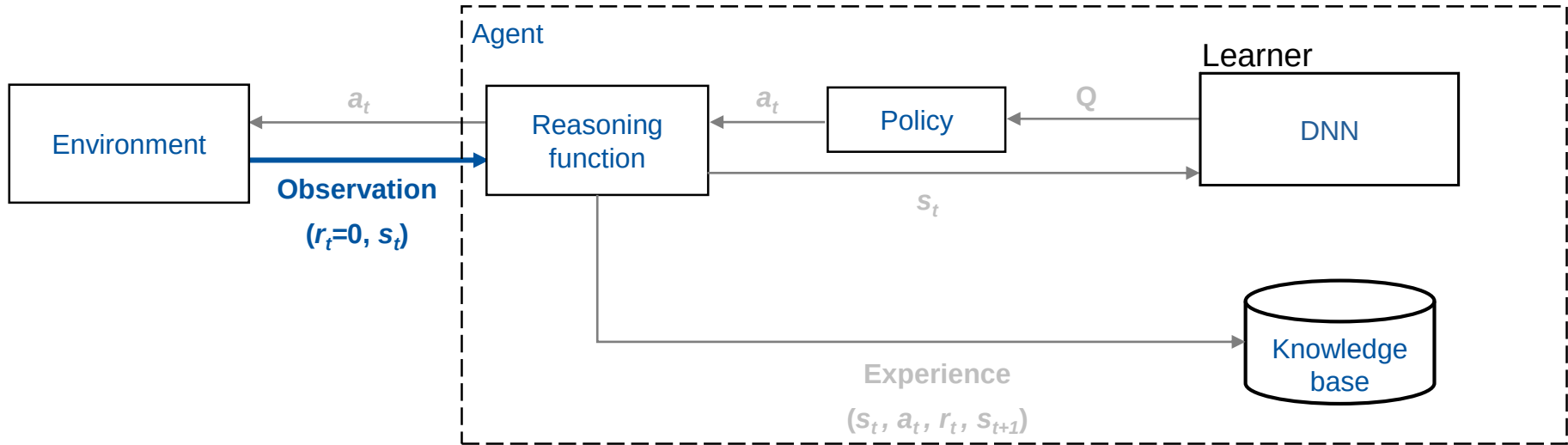
Deep RL agent

- The agent uses DNN to capture the information of the “infinite” state space



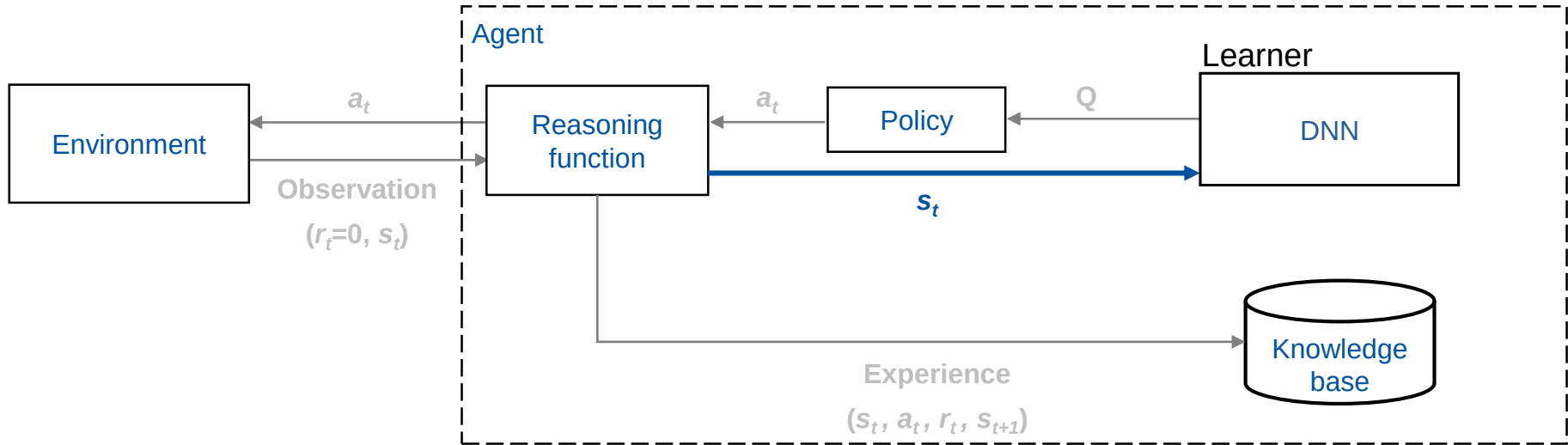
Warmup phase

- Initially the environment sends the current state (at $t = 0$, the state is s_0 and the reward is 0)



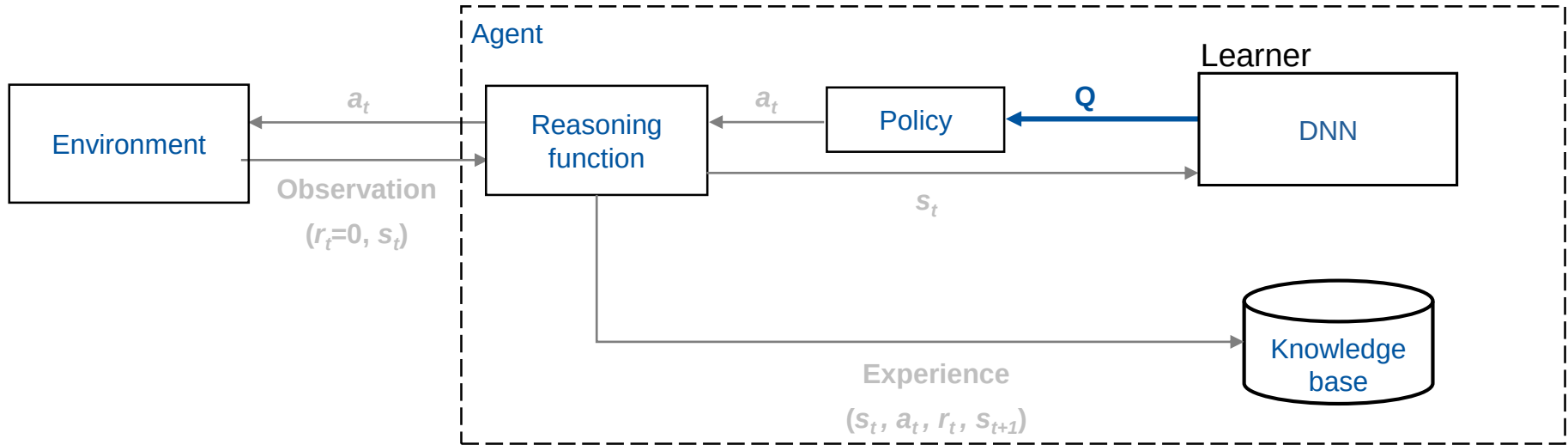
Warmup phase

- The Reasoning function push the current state to the DNN.



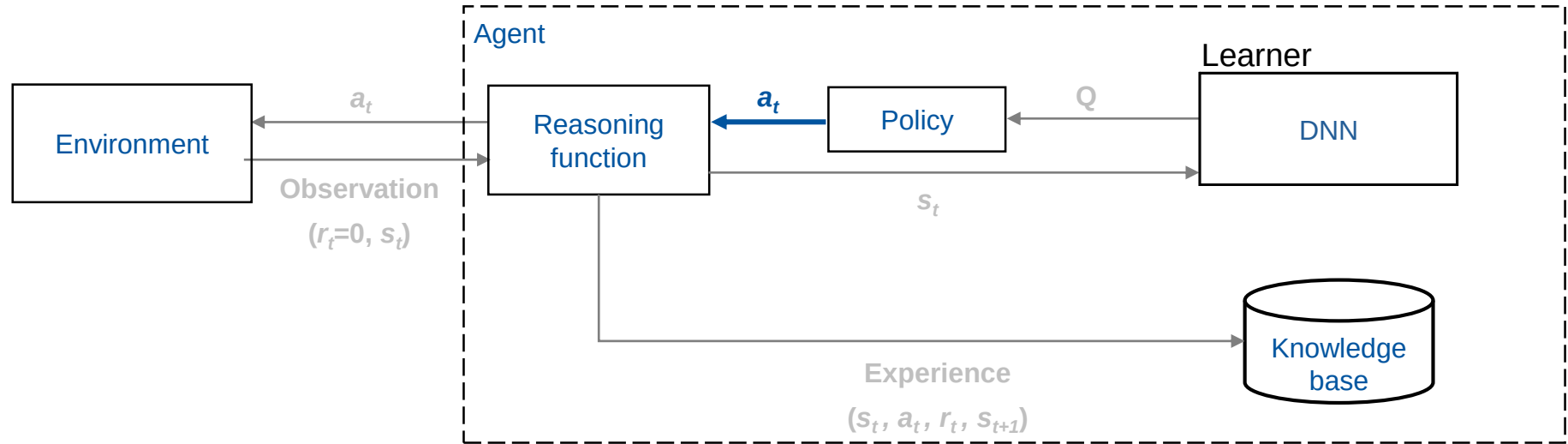
Warmup phase

- DNN predicts the corresponding Q values.



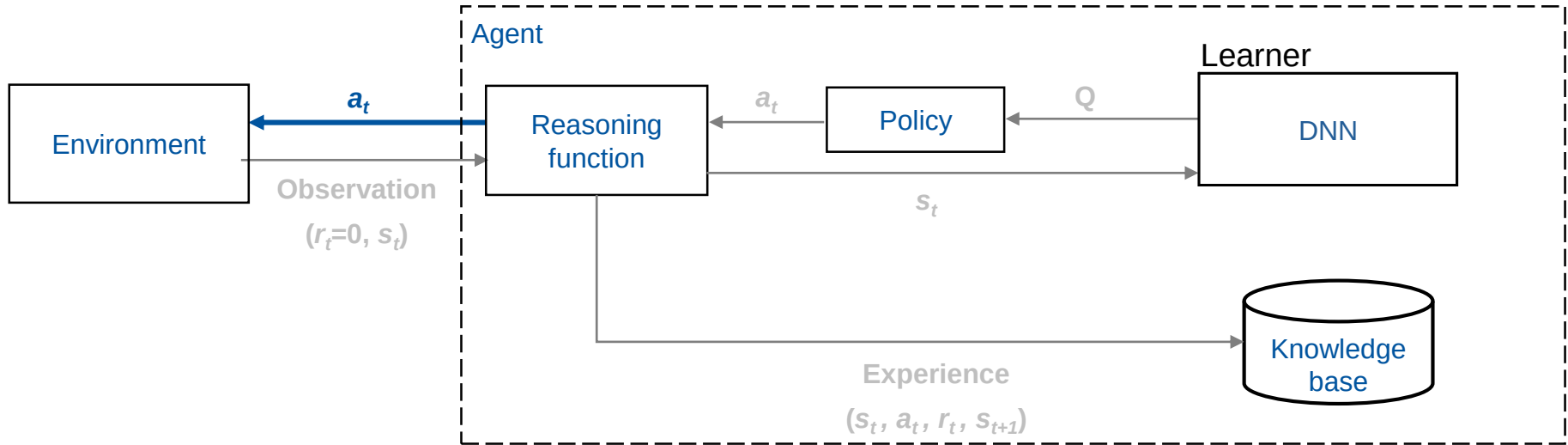
Warmup phase

- Depending on the **exploration policy** an action is taken considering the Q values



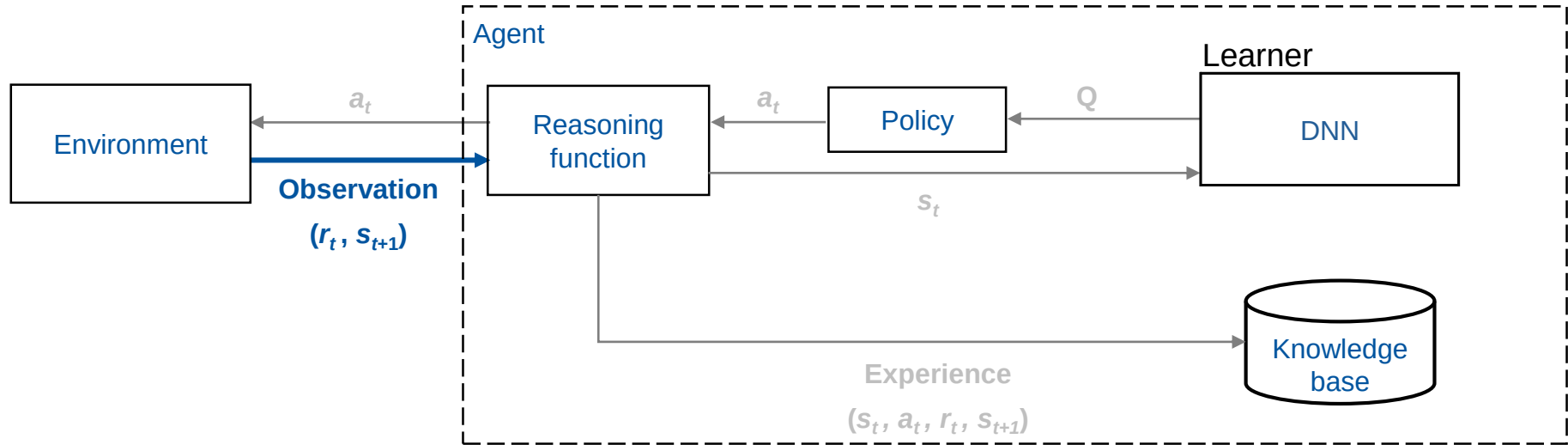
Warmup phase

- The reasoning triggers the action execution in the environment



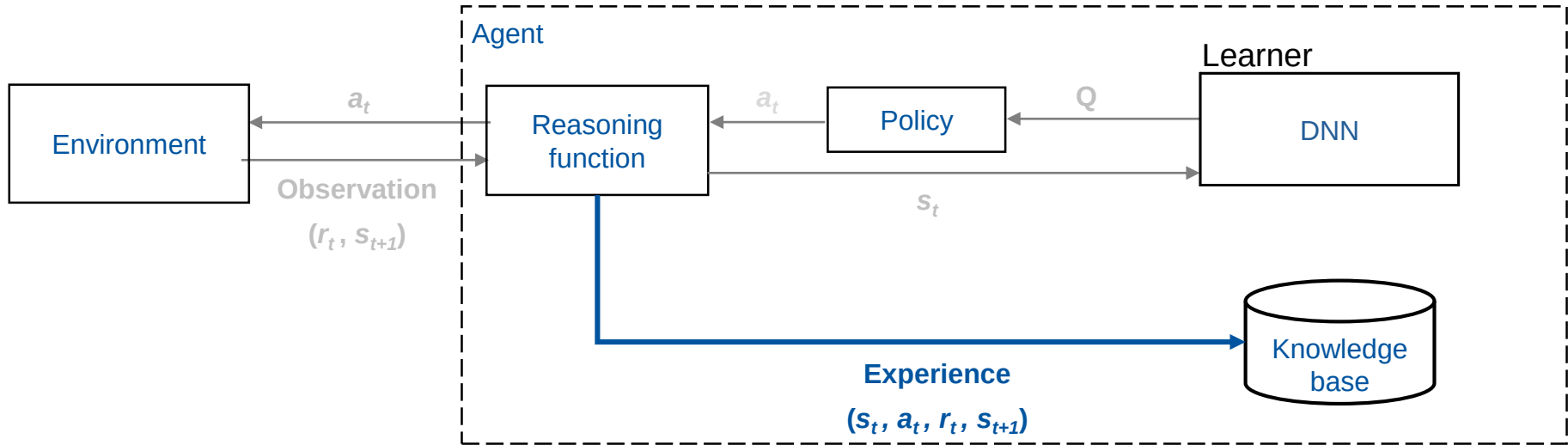
Warmup phase

- The Reasoning function receives the reward for taken action and the next state of the environment



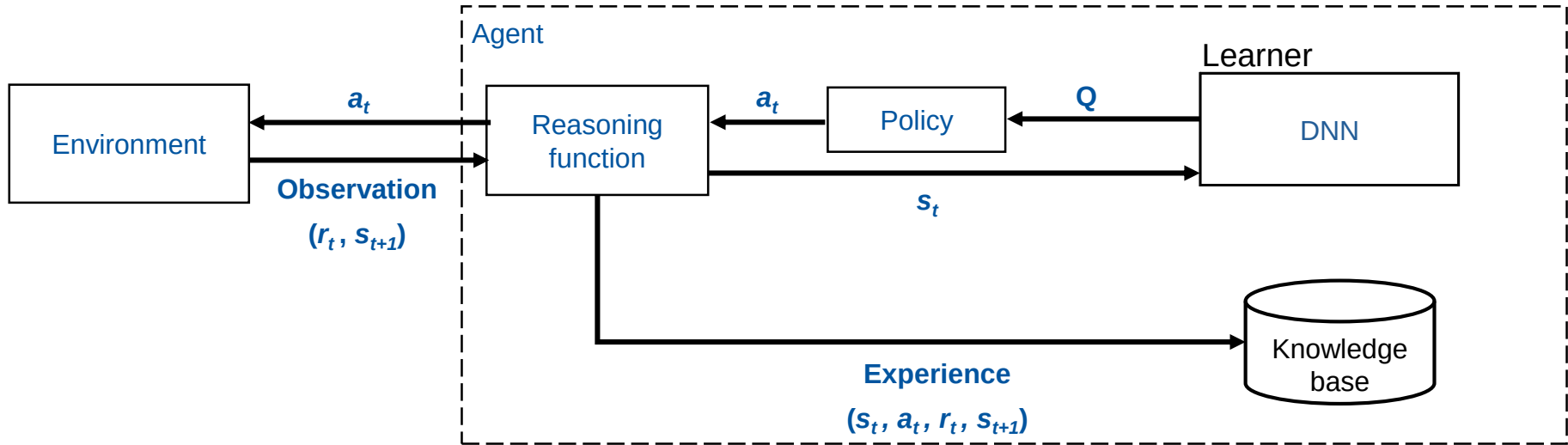
Warmup phase

- The reasoning function saves the states, actions and rewards in the memory for future use (training)



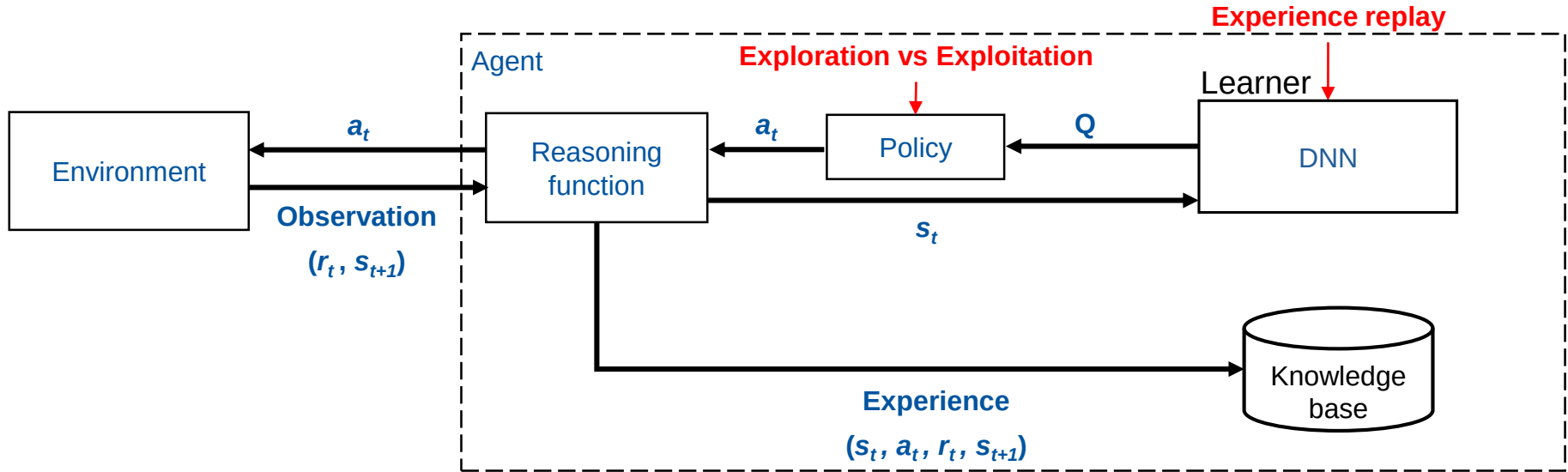
Warmup phase

- The prediction cycle continues further on as illustrated
- Knowledge base has a finite size so only a number of latest sequences are saved



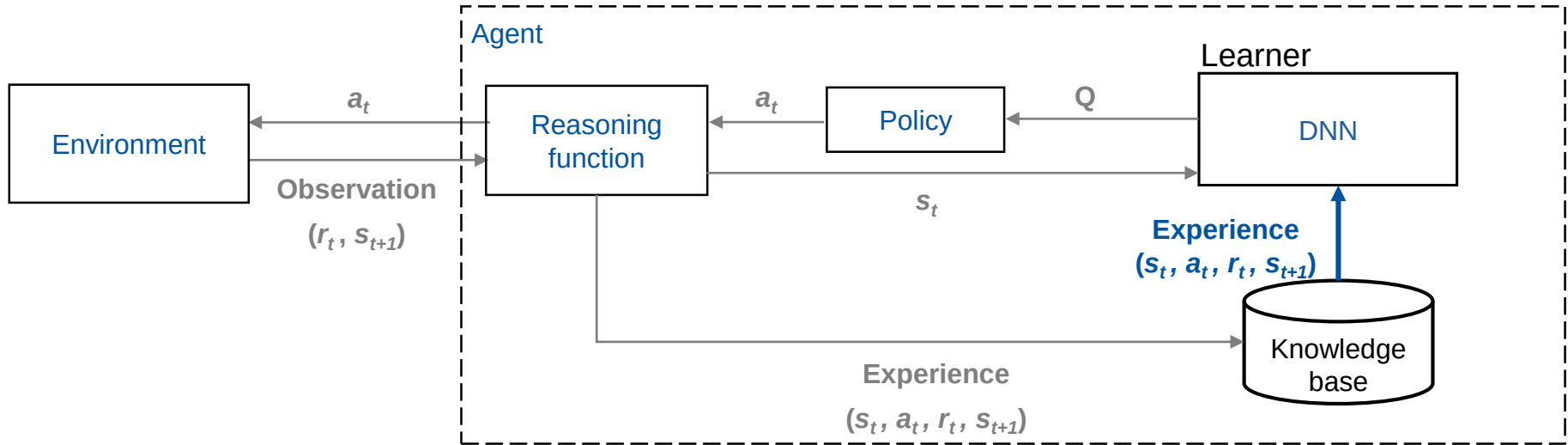
Training phase

- Training phase = Exploration + Exploitation + Experience replay
 - The best action with the maximum Q value is taken following the exploration policy
 - Based on the fixed Q-targets the Experience replay is designed to train DNN



Training phase – Experience replay

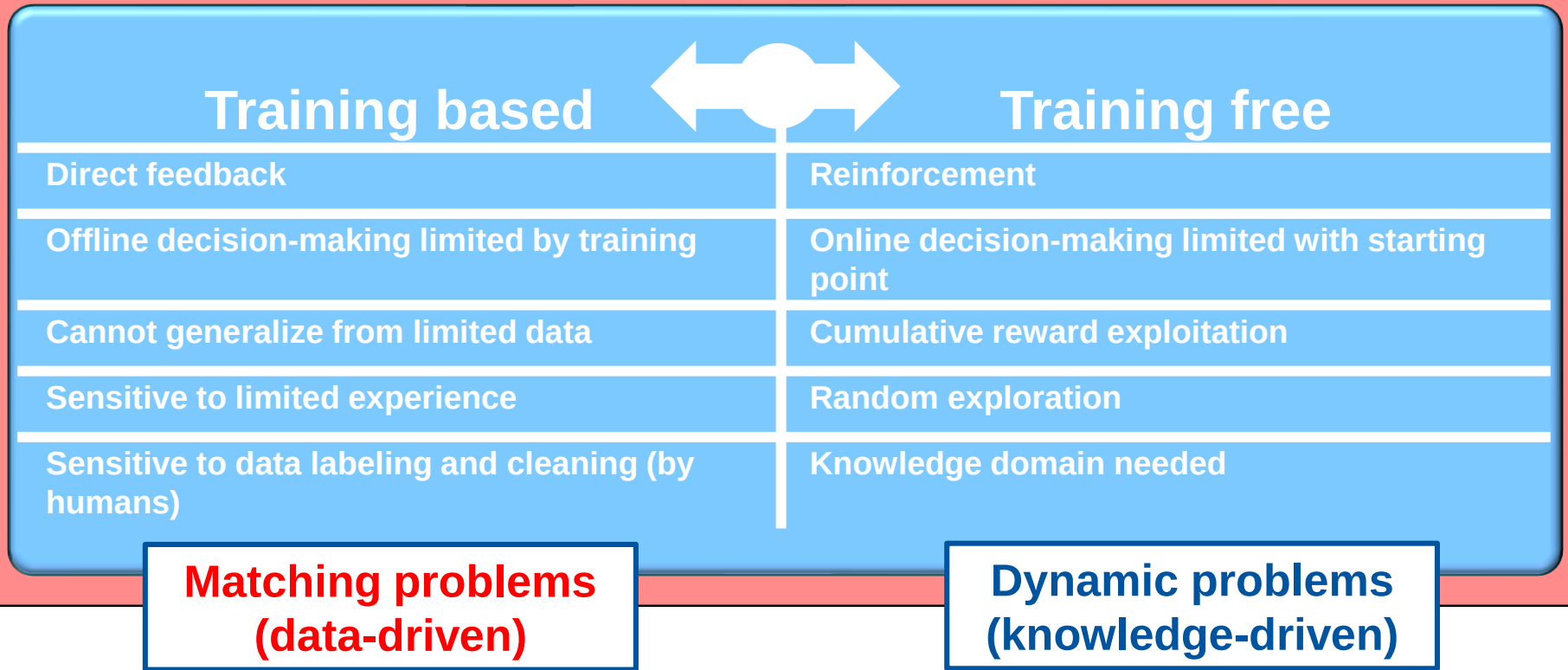
- DNN training is done using a process known as **experience replay**
- Initially the DNN is untrained so the **Q values are random**
- **Experience replay**: The process of training so that the Q-values are meaningful



Remarks

- One of the biggest advantages of using deep learning approach is its ability to execute feature engineering by itself.
- Ability to compactly represent both high-dimensional observations and the Q-function using deep neural networks
- The only difference between Q-learning and DQN is the agent's brain
 - in Q-learning it is the Q-table
 - in DQN it is a deep neural network
- Although Q-learning is a very powerful algorithm, its main weakness is lack of generality.

General remarks



End of the course

